

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY

GURAJADA VIZIANAGARAM

VIZIANAGARAM – 533003

ANDHRA PRADESH

STUDENT HELP COMMUNITY APP

By

METTA BHARGAV

(Regd. No. 24VV1A1241)

Under the supervision of

MRS. MADHUMITHA CHANDA

MRS. M. GEETHA MADHURI YADAV

Department of Information Technology

Vizianagaram – 533003



JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Regd.No : 24VV1A1241

CERTIFICATE

This is to certify that this is a bonafide record of practical work done by **METTA BHARGAV** of IIIrd B.Tech Ist Semester Class in User Interface Design using Flutter Lab during the year 2026-27.

No.of Tasks Completed and Certified: 11

Lecture In-Charge **Head of The Department**

Date:

DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Website:www.jntugvcev.edu.in

Subject Name: USER INTERFACE DESIGN USING FLUTTER Subject Code: R233112ES01

Academic Year: 2026 **Regulation: R23**

COURSE OUTCOMES

NBA Subject Code	Course Outcomes	
R233112ES01	CO1	Learns to Implement Flutter Widgets and Layouts.
	CO2	Understands Responsive UI Design and with Navigation in Flutter.
	CO3	Knowledge on Widgets and customize widgets for specific UI elements, Themes
	CO4	Understand to include animation apart from fetching data.

CO-PO Mapping

Mapping of Course Outcomes (COs) with Program Outcomes (POs)

Course Outcomes		Program Outcomes (POs)														
		PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
R233112ES01	CO1	3	2	3	2	3	1	1	1	2	2	1	2	3	3	2
	CO2	3	3	3	2	3	1	2	2	2	3	2	2	3	3	2
	CO3	3	2	3	2	3	2	1	1	2	2	1	2	3	2	2
	CO4	3	3	3	3	3	2	1	2	3	3	2	3	3	3	3

Enter correlation levels 1,2 and 3 as defined below:

1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High) If there is no correlation, put "-"

Signature of the Course Instructor

CONTENTS

S.No.	Date	Experiment Name	Page No.	Marks	Signature
1(a)		Installation of Flutter and Dart SDK	1		
1(b)		Dart Programming Basics	2		
2(a)		Explore Flutter Widgets	3		
2(b)		Layouts Using Row, Column, and Stack	5		
3(a)		Responsive User Interface	7		
3(b)		Media Queries and Breakpoints	9		
4(a)		Navigation Using Navigator	11		
4(b)		Navigation Using Named Routes	13		
5(a)		Stateful and Stateless Widgets	14		
5(b)		State Management Using setState	15		
6(a)		Creating Custom Widgets	16		
6(b)		Themes and Custom Styles	17		
7(a)		Creating Forms with Input Fields	18		
7(b)		Form Validation and Error Handling	20		
8(a)		Implementing Animations	22		
8(b)		Different Animation Types	23		
9(a)		Connecting to a REST API	25		
9(b)		Displaying Fetched API Data	26		
10(a)		Unit Testing in Flutter	27		
10(b)		Debugging a Flutter Application	28		
11		Case Study: Student Help Community	29		

Experiment 1(a): Install Flutter and Dart SDK

Aim

To install and verify the Flutter framework and Dart SDK required to develop and run Student Help Community.

Description

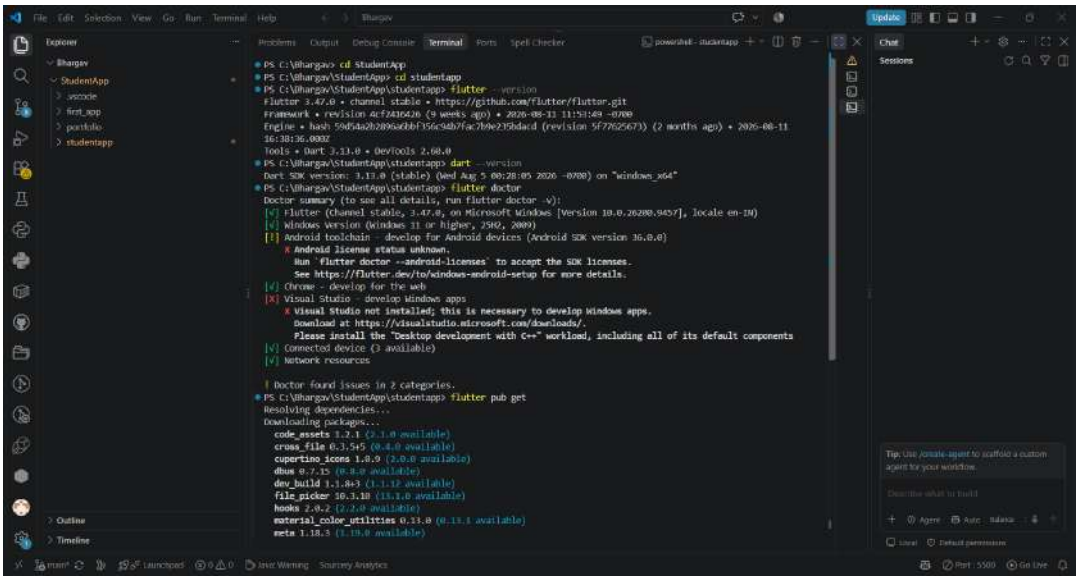
Flutter is a cross-platform UI toolkit and Dart is the programming language used to create the application's widgets, screens and business logic. The Flutter command-line tools verify the installation, resolve package dependencies, identify available devices and launch the app on Android or Chrome. The output for this experiment should be captured from the actual development computer.

Code

```
flutter --version
dart --version
flutter doctor
flutter pub get
flutter devices
flutter run -d chrome
# For Android, use the device ID shown by flutter devices:
flutter run -d <android-device-id>
```

Output

The terminal displays Flutter and Dart versions, environment diagnostics, installed dependencies and available target devices.



Experiment 1(b): Write a simple Dart program to understand language basics

Aim

To understand Dart variables, data types, strings, lists and basic output using examples related to Student Help Community.

Description

Dart variables store values used by an application. A student community can represent a student's name, a question title, a number of answers and a list of resource categories using String, int and List types. The following small console program is a standalone Dart language exercise; it does not replace the Flutter application.

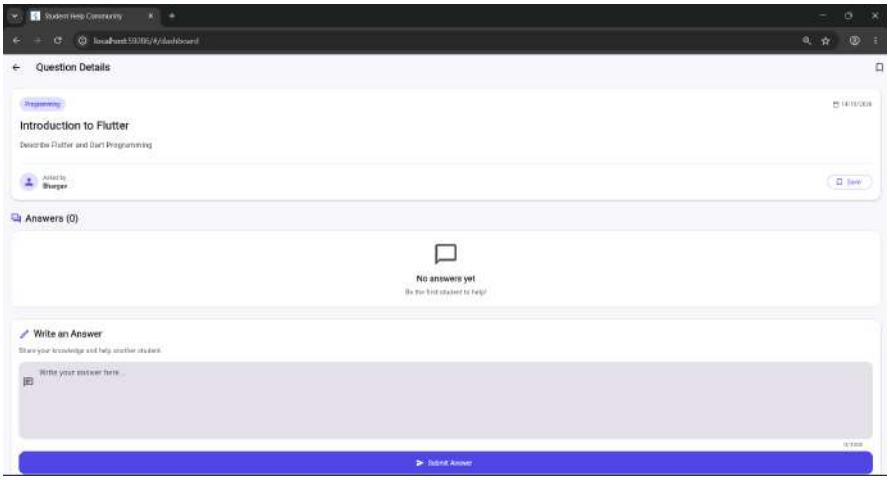
Code

```
void main() {
  String studentName = 'Student';
  String questionTitle = 'How does inheritance work?';
  int answerCount = 3;
  List<String> resourceCategories = [
    'Programming', 'Data Structures', 'Database'
  ];

  print('Student Help Community');
  print('Student: $studentName');
  print('Question: $questionTitle');
  print('Answers: $answerCount');
  print('Categories: ${resourceCategories.join(', ')}');
}
```

Output

The console prints the application title, a sample student name, a sample question, the answer count and resource categories.



Experiment 2(a): Explore Flutter widgets

Aim

To explore common Flutter widgets used to construct Student Help Community screens.

Description

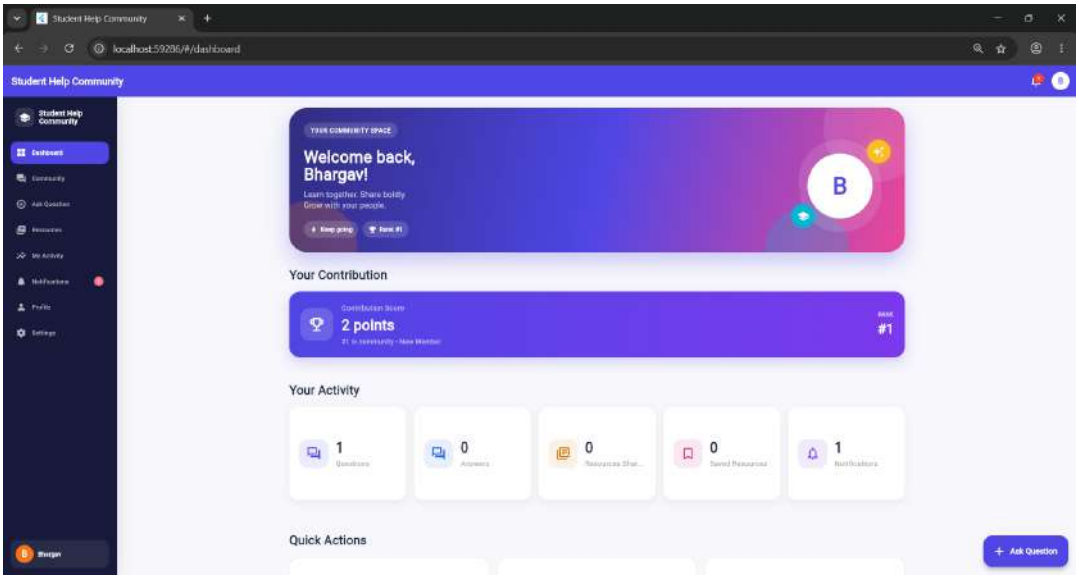
Flutter interfaces are built from widgets. Text displays headings and question titles; Icon provides visual cues; Card groups related information; Container applies spacing and decoration; TextField collects user input; ElevatedButton triggers actions; ListTile represents a question or resource row. The real app uses combinations of these building blocks throughout login, dashboard, question and resource screens.

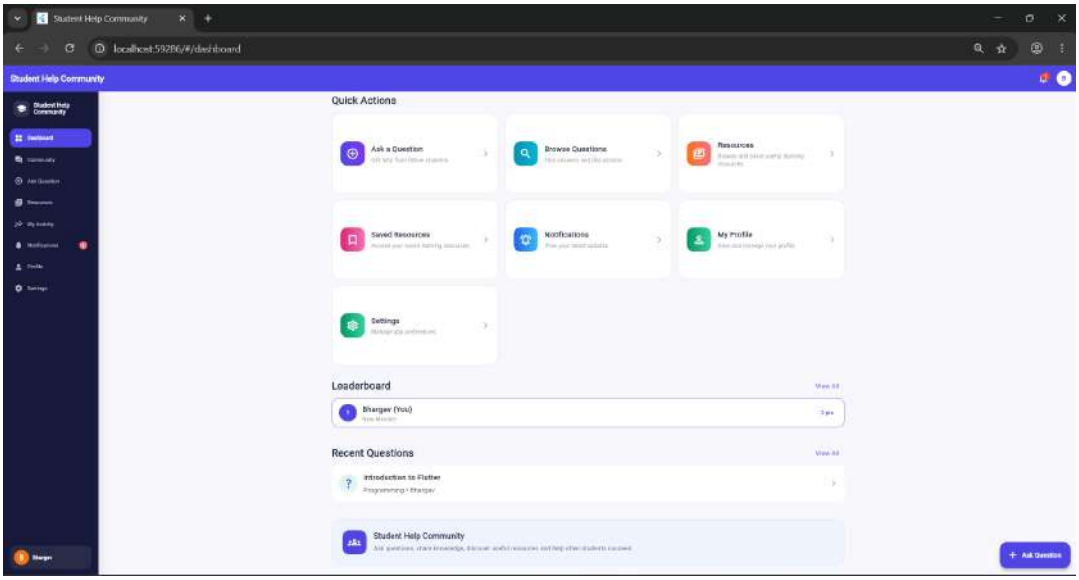
Code

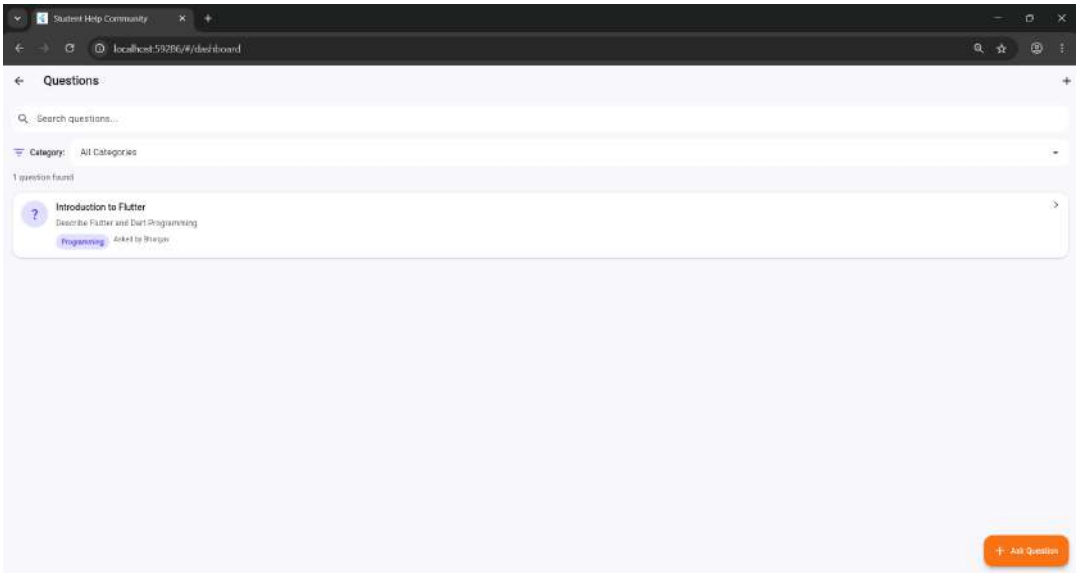
```
// Small widget example based on a community question card.
Card(
  child: ListTile(
    leading: const Icon(Icons.help_outline),
    title: const Text('How does inheritance work?'),
    subtitle: const Text('Programming | 3 answers'),
    trailing: const Icon(Icons.arrow_forward_ios, size: 16),
  ),
)
```

Output

A card displays a sample academic question, category, answer count and navigation icon.




Student Help Community
localhost:59286/#/dashboard
Student Help Community
Quick Actions
Ask a Question
Browse Questions
Resources
Saved Resources
Notifications
My Profile
Settings
Leaderboard
Recent Questions
Introduction to Flutter
Student Help Community
+ Ask Question


Student Help Community
localhost:59286/#/dashboard
Questions
Search questions...
Category:
All Categories
1 question found
Introduction to Flutter
Describe Flutter and Dart Programming
Programming
+ Ask Question

Experiment 2(b): Implement layouts using Row, Column and Stack

Aim

To demonstrate horizontal, vertical and layered layouts using components relevant to Student Help Community.

Description

Row places child widgets horizontally, Column places them vertically and Stack layers widgets in the same area. A dashboard statistic can use Row for an icon and value, Column for the label and number, and Stack can position a small notification badge over an icon. The following is a learning example; match its appearance to a real dashboard or question card when capturing output.

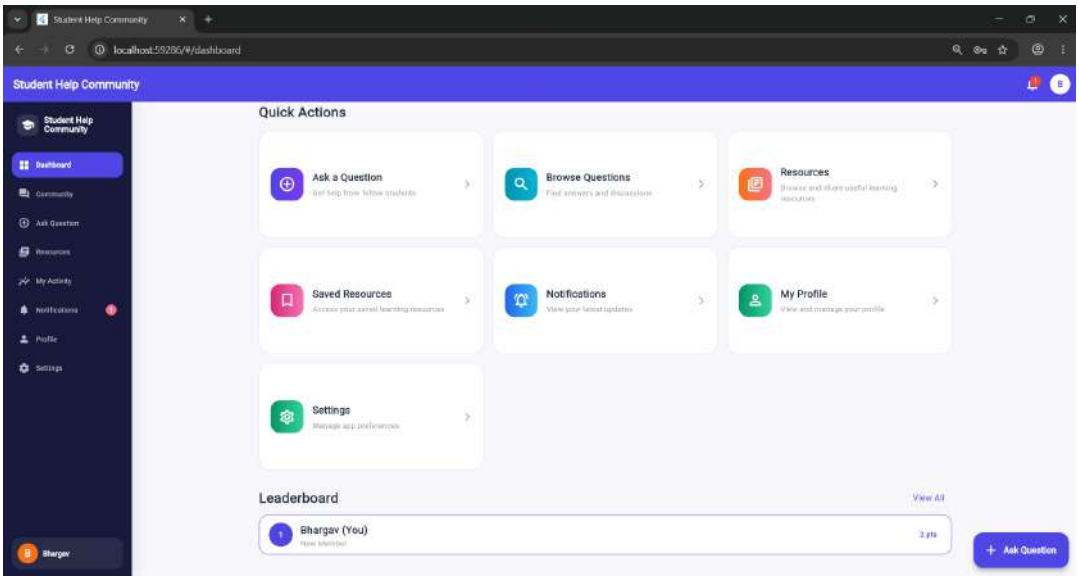
Code

```
Card(
  child: Padding(
    padding: const EdgeInsets.all(16),
    child: Row(
      children: [
        Stack(
          children: [
            const CircleAvatar(
              child: Icon(Icons.forum_outlined),
            ),
            Positioned(
              right: 0, top: 0,
              child: Container(
                width: 9, height: 9,
                decoration: const BoxDecoration(
                  color: Colors.green,
                  shape: BoxShape.circle,
                ),
              ),
            ),
          ],
        ),
        const SizedBox(width: 12),
        const Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text('Questions'),
            Text('Ask and learn together'),
          ],
        ),
```

```
      ],
    ),
  ),
)
```

Output

The example places a community icon and text side by side, arranges the text vertically and overlays a small status dot using Stack.



Experiment 3(a): Design a responsive UI for different screen sizes

Aim

To demonstrate a responsive Student Help Community interface that adapts to a mobile screen and a wider Chrome browser.

Description

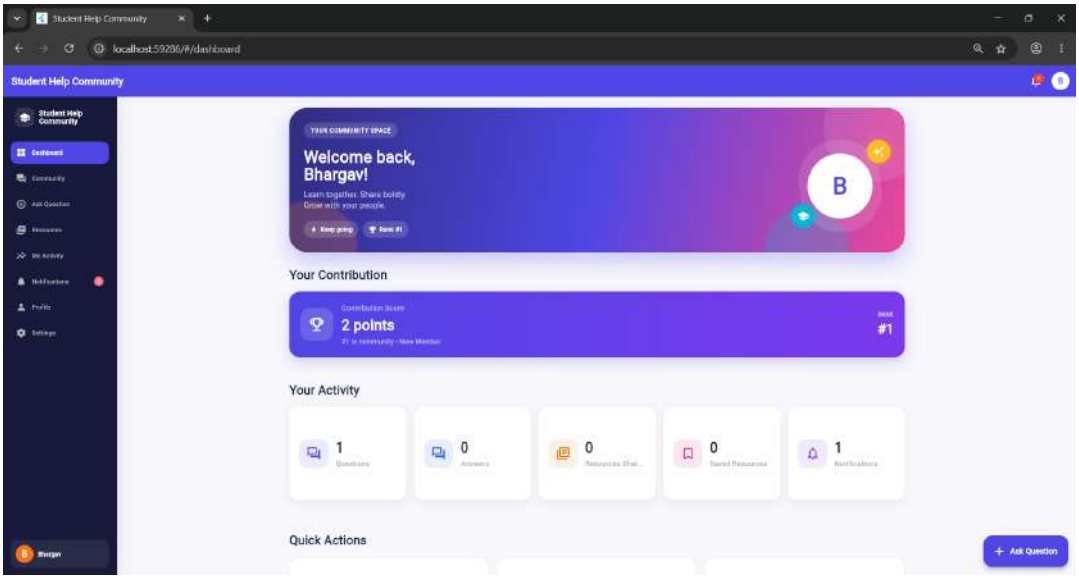
Responsive design adapts the number of columns, spacing and navigation to the available width. A mobile device can show a single-column dashboard, while a wider browser can show several dashboard cards side by side. LayoutBuilder allows Flutter to inspect the space available to a widget. The code is a reusable example and must be aligned with the final layout used in the app.

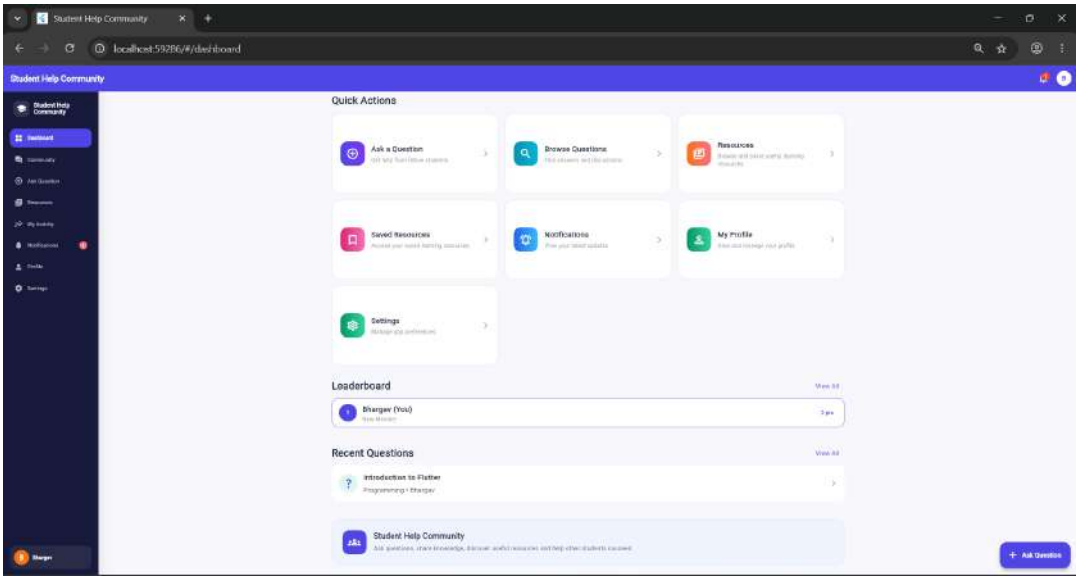
Code

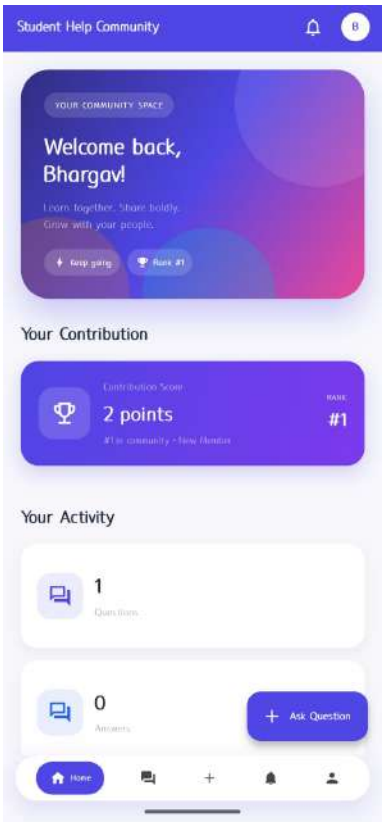
```
LayoutBuilder(
  builder: (context, constraints) {
    final isWide = constraints.maxWidth >= 700;
    return GridView.count(
      crossAxisCount: isWide ? 3 : 1,
      crossAxisSpacing: 12,
      mainAxisSpacing: 12,
      children: communityCards,
    );
  },
)
```

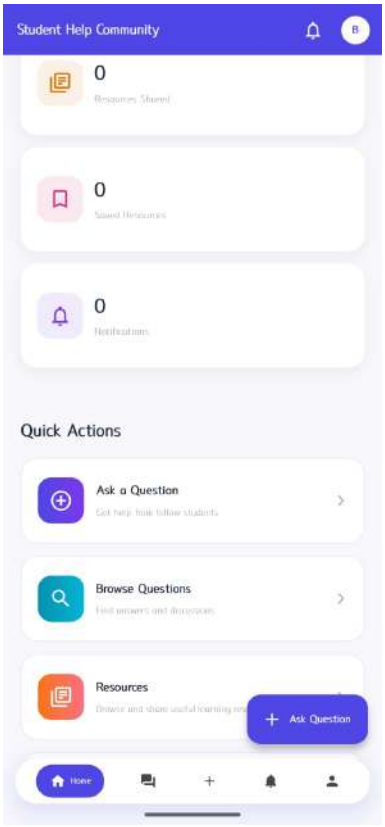
Output

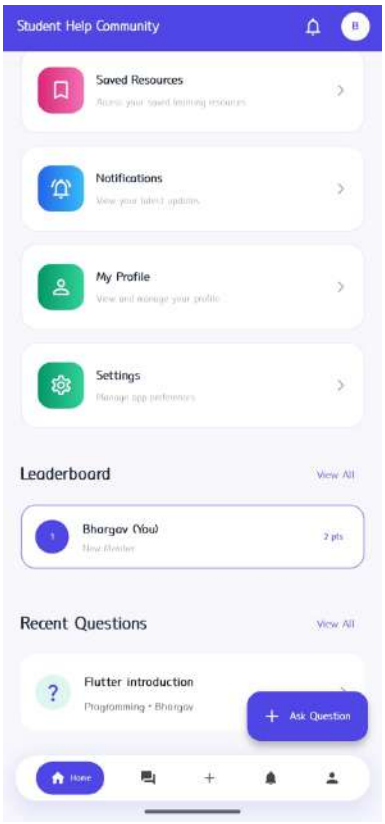
The same dashboard or resource list should remain readable on a narrow Android screen and a wide Chrome window.




Student Help Community
localhost:59286/#/dashboard
Student Help Community
Quick Actions
Ask a Question
Browse Questions
Resources
Saved Resources
Notifications
My Profile
Settings
Leaderboard
Recent Questions
Student Help Community
+ Ask Question


Student Help Community
YOUR COMMUNITY SPACE
Welcome back, Bhargav!
Learn together. Share boldly. Grow with your people.
Keep going
Rank #1
Your Contribution
Contribution Score
2 points
RANK
#1
Your Activity
1
Questions
0
Answers
+ Ask Question
Home


Student Help Community
0
Resources Shared
0
Saved Resources
0
Notifications
Quick Actions
Ask a Question
Get help from fellow students
Browse Questions
Find answers and discussions
Resources
Browse and share useful learning resources
+ Ask Question
Home


Student Help Community
Saved Resources
Access your saved learning resources
Notifications
View your latest updates
My Profile
View and manage your profile
Settings
Manage app preferences
Leaderboard
View All
Bhargav (You)
New Member
2 pts
Recent Questions
View All
Flutter introduction
Programming • Bhargav
+ Ask Question
Home

Experiment 3(b): Implement media queries and breakpoints for responsiveness

Aim

To use MediaQuery and breakpoints to adjust the Student Help Community layout according to screen width.

Description

Flutter uses MediaQuery and LayoutBuilder rather than CSS media queries. A breakpoint can change the number of columns used for dashboard cards or resource tiles. The thresholds below are sample values and should be tuned by testing the actual app on a phone and a browser.

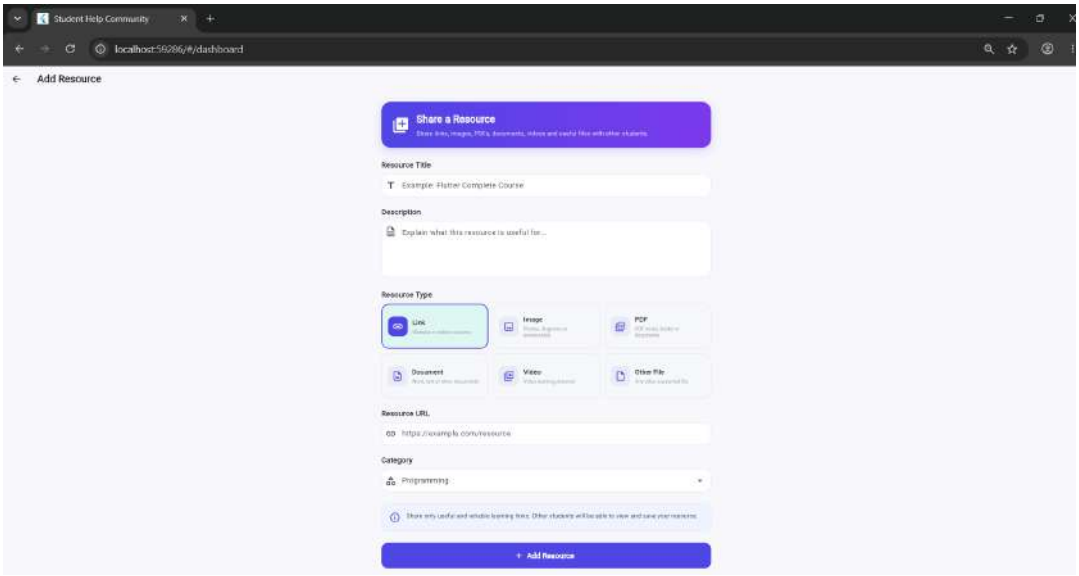
Code

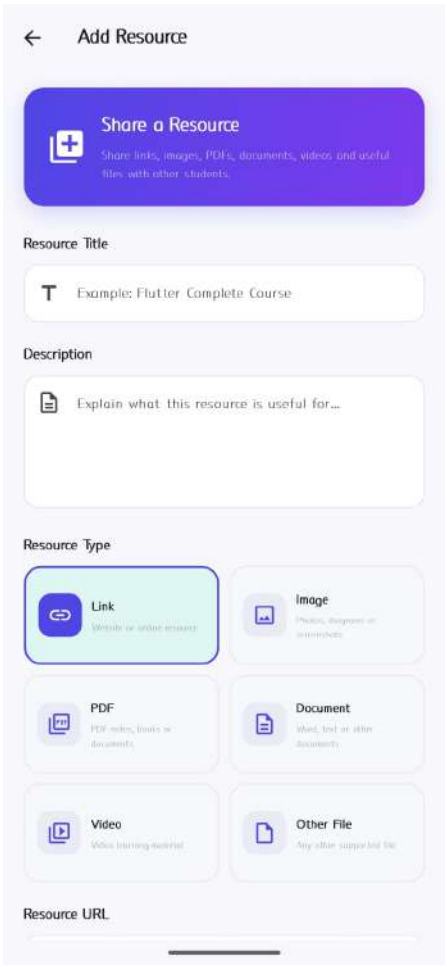
```
final width = MediaQuery.sizeOf(context).width;
final columns = width >= 1000 ? 4 : (width >= 650 ? 2 : 1);

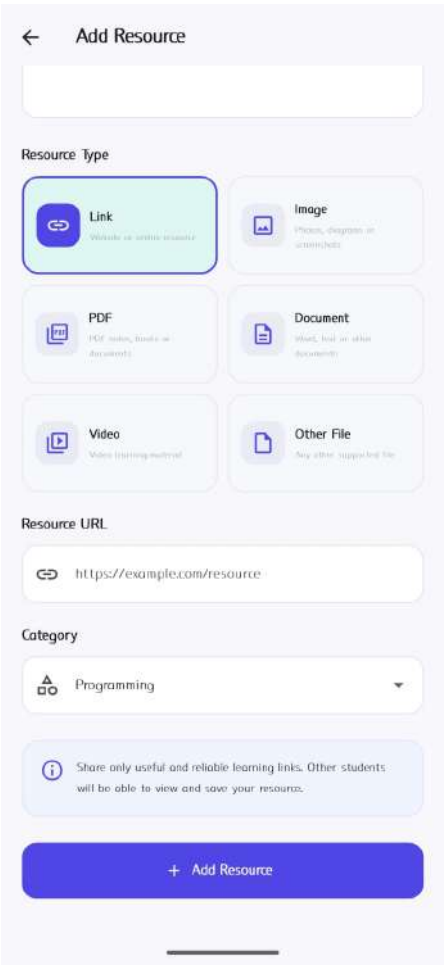
return GridView.count(
  crossAxisCount: columns,
  crossAxisSpacing: 12,
  mainAxisSpacing: 12,
  children: communityCards,
);
```

Output

Capture the same resource grid at a narrow mobile width and a wide browser width.




Add Resource
Share a Resource
Share links, images, PDFs, documents, videos and useful files with other students.
Resource Title
Example: Flutter Complete Course
Description
Explain what this resource is useful for...
Resource Type
Link
Image
PDF
Document
Video
Other File
Resource URL


Add Resource
Resource Type
Link
Image
PDF
Document
Video
Other File
Resource URL
https://example.com/resource
Category
Programming
Share only useful and reliable learning links. Other students will be able to view and save your resource.
Add Resource

Experiment 4(a): Set up navigation between screens using Navigator

Aim

To navigate between Student Help Community screens using Flutter Navigator.

Description

Navigator maintains a route stack. Navigator.push opens a destination and Navigator.pop returns to the previous screen. In the app workflow, a user can open Ask Question from the dashboard, open a question from the Questions list and return to the previous screen. Use the real screen class names from the project when integrating the example.

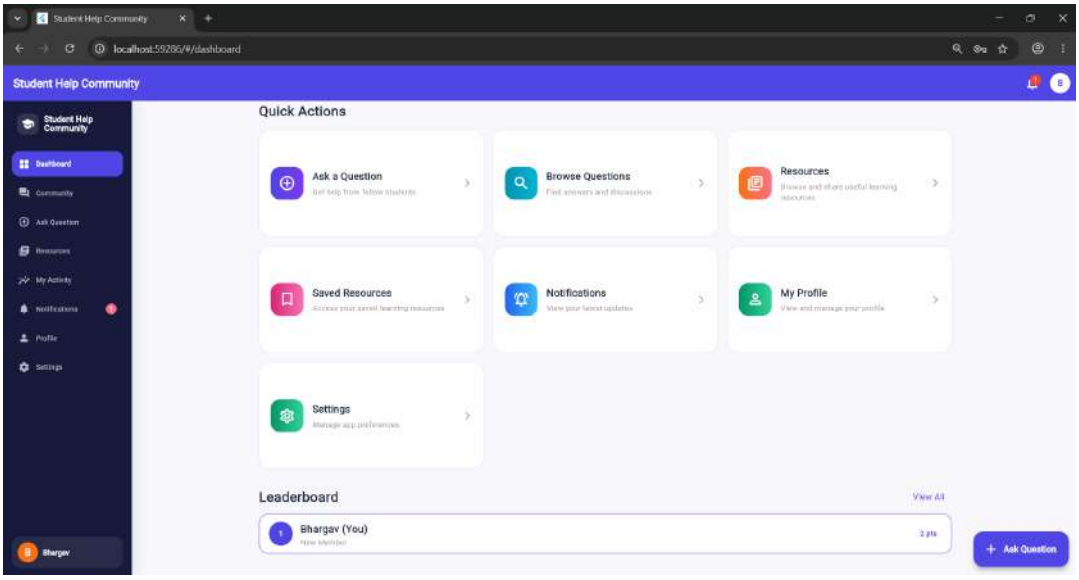
Code

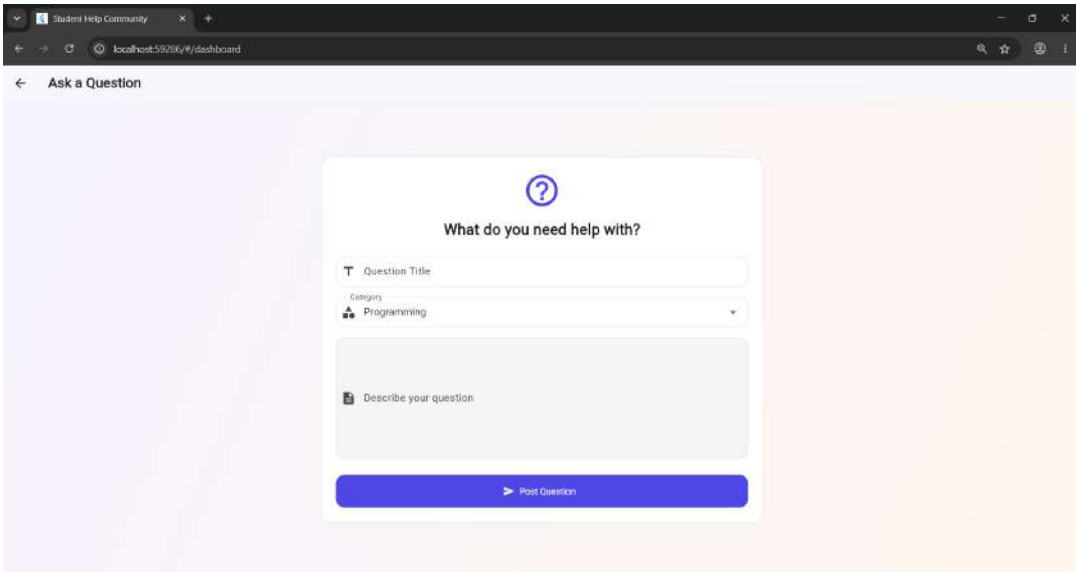
```
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => const AskQuestionScreen(),
  ),
);

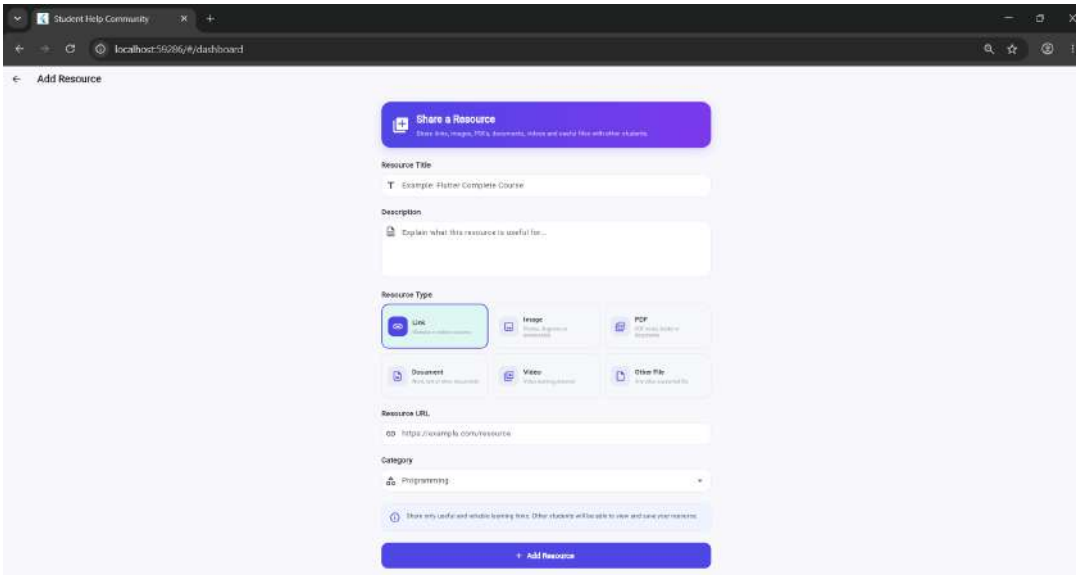
// Return to the previous route when appropriate:
Navigator.pop(context);
```

Output

Show the dashboard or question list before navigation and the destination Ask Question screen after selecting the relevant action.




Student Help Community
localhost:59286/#/dashboard
Ask a Question
What do you need help with?
Question Title
Category
Programming
Describe your question
Post Question


Student Help Community
localhost:59286/#/dashboard
Add Resource
Share a Resource
Resource Title
Description
Resource Type
Link
Image
PDF
Document
Video
Other File
Resource URL
Category
Programming
Add Resource

Experiment 4(b): Implement navigation with named routes

Aim

To demonstrate named-route navigation using route names for Student Help Community screens.

Description

Named routes assign string identifiers to destination screens and can make navigation calls consistent across an application. The currently implemented app should only be described as using named routes if its MaterialApp actually declares a routes table or onGenerateRoute. Otherwise, this is a separate lab demonstration of the syllabus concept.

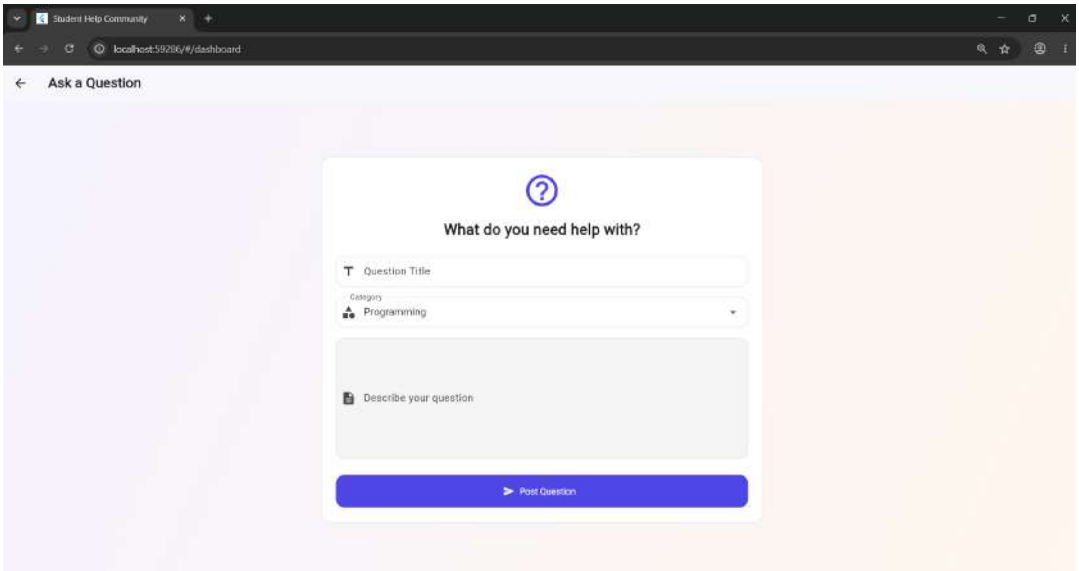
Code

```
// Demonstration: configure these routes in MaterialApp.
MaterialApp(
  routes: {
    '/dashboard': (context) => const DashboardScreen(),
    '/ask-question': (context) => const AskQuestionScreen(),
  },
);

// Open the named destination:
Navigator.pushNamed(context, '/ask-question');
```

Output

The Ask Question screen opens when the named route is triggered. Capture the real destination screen. If the existing app uses MaterialPageRoute instead, label this as a standalone named-route experiment.



Experiment 5(a): Learn about StatefulWidget and StatelessWidget

Aim

To understand the difference between stateless and stateful widgets using Student Help Community UI examples.

Description

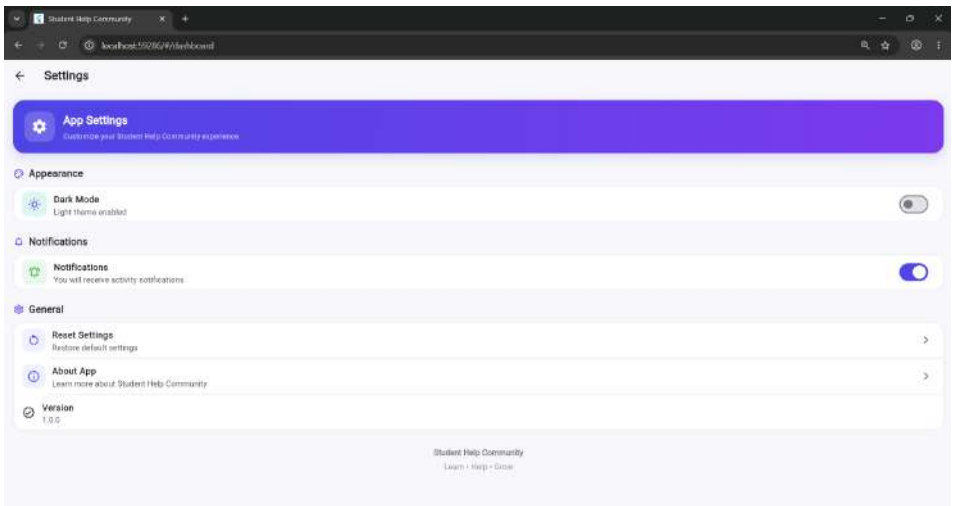
A StatelessWidget is suitable for UI that depends only on its inputs, such as a reusable section heading or category label. A StatefulWidget is suitable when values change during interaction, such as text entered in a form, a selected filter or a visibility toggle.

Code

```
class CommunitySectionTitle extends StatelessWidget {
  const CommunitySectionTitle({super.key, required this.title});
  final String title;
  @override
  Widget build(BuildContext context) => Text(
    title,
    style: Theme.of(context).textTheme.titleLarge,
  );
}
class ExampleCounter extends StatefulWidget {
  const ExampleCounter({super.key});
  @override
  State<ExampleCounter> createState() => _ExampleCounterState();
}
```

Output

Use a real screen containing static headings and a changing control or selection.



Experiment 5(b): Implement state management using setState and Provider

Aim

To demonstrate state updates in Student Help Community and understand local and shared state management.

Description

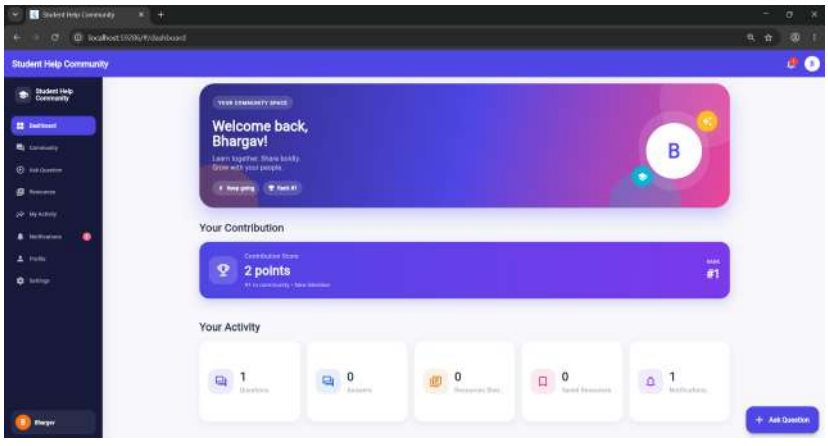
setState tells Flutter that a StatefulWidget's local state has changed and its UI should be rebuilt. It can update a selected category, a toggle or a local counter. Provider is an optional package for exposing shared state to multiple widgets.

Code

```
class _CategorySelectorState extends State<CategorySelector> {
  String selectedCategory = 'Programming';
  @override
  Widget build(BuildContext context) {
    return DropdownButton<String>(
      value: selectedCategory,
      items: const ['Programming', 'Data Science', 'Database']
          .map((item) => DropdownMenuItem(
                value: item, child: Text(item)))
          .toList(),
      onChanged: (value) {
        if (value == null) return;
        setState(() => selectedCategory = value);
      },
    );
  }
}
```

Output

Select a different category and capture the updated value on screen.



Experiment 6(a): Create custom widgets for specific UI elements

Aim

To create reusable custom widgets suitable for Student Help Community screens.

Description

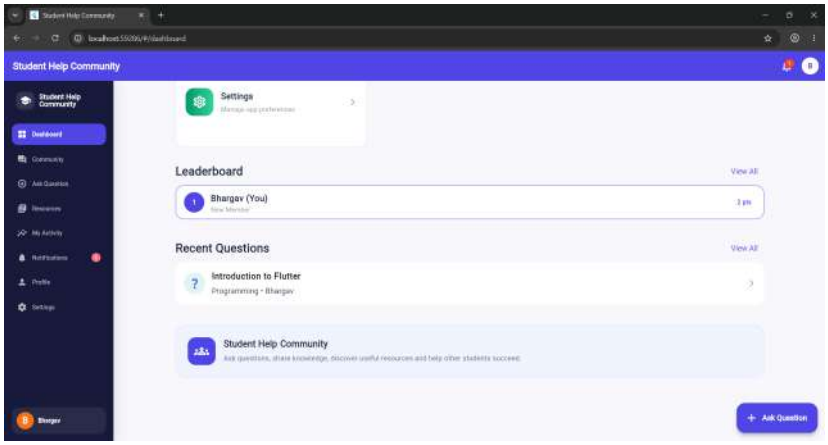
Custom widgets reduce repeated code and provide consistent visual styling. A custom widget can accept parameters such as title, subtitle, icon and tap callback. The sample below shows a reusable question card pattern.

Code

```
class CommunityQuestionCard extends StatelessWidget {
  const CommunityQuestionCard({
    super.key,
    required this.title,
    required this.subtitle,
    this.onTap,
  });
  @override
  Widget build(BuildContext context) => Card(
    child: ListTile(
      leading: const Icon(Icons.question_answer_outlined),
      title: Text(title),
      subtitle: Text(subtitle),
      onTap: onTap,
    ),
  );
}
```

Output

Capture the Questions screen or dashboard where question cards are visible.



Experiment 6(b): Apply styling using themes and custom styles

Aim

To apply a consistent theme and custom styling to Student Help Community.

Description

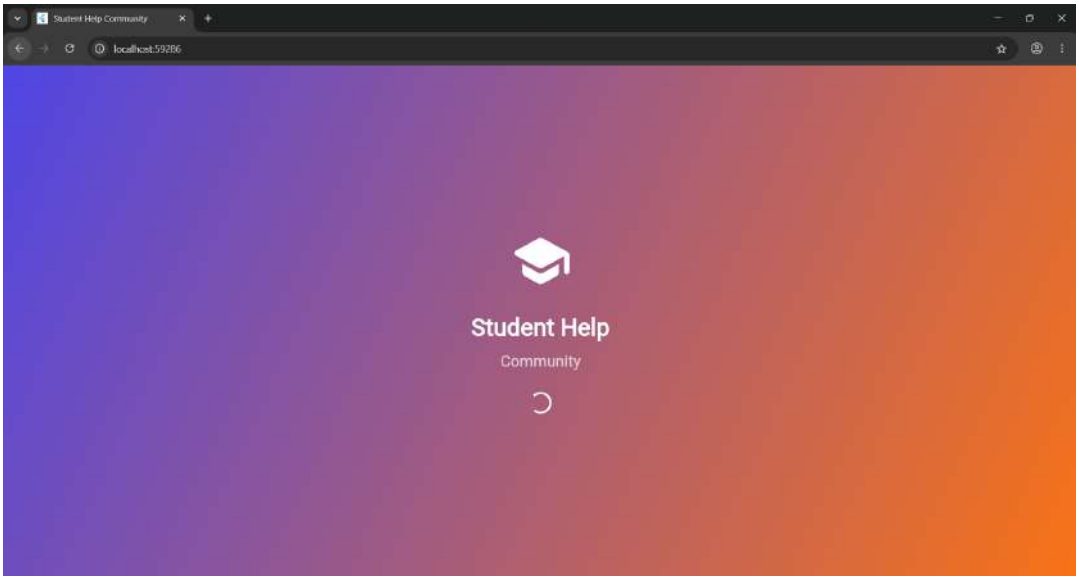
ThemeData centralizes colors, typography and common component defaults. A consistent theme improves readability and creates a recognizable visual identity across login, dashboard, questions, resources, profile and admin screens.

Code

```
MaterialApp(
  theme: ThemeData(
    useMaterial3: true,
    colorScheme: ColorScheme.fromSeed(
      seedColor: const Color(0xFF6750A4),
    ),
    cardTheme: const CardThemeData(
      elevation: 2,
      margin: EdgeInsets.symmetric(vertical: 6),
    ),
  ),
  home: const SplashScreen(),
)
```

Output

Capture the actual themed splash screen.



Experiment 7(a): Design a form with various input fields

Aim

To design a student community form with text fields, category selection and a submission action.

Description

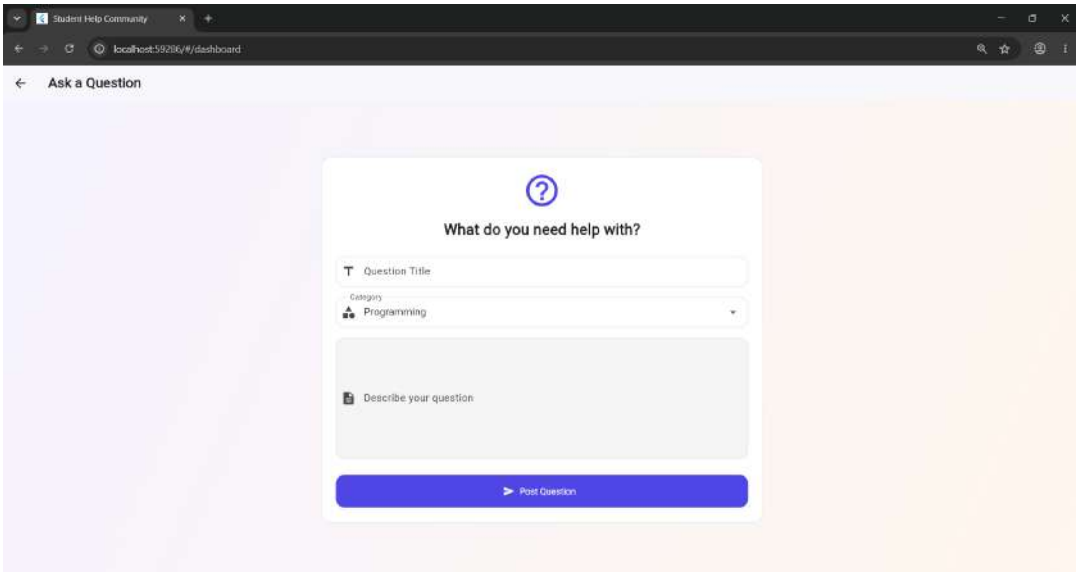
Forms collect structured user input. The Ask Question screen may collect a question title, detailed description and category. Resource sharing can collect a title, description, category and link or file. TextEditingController values should be read and disposed of appropriately in stateful screens.

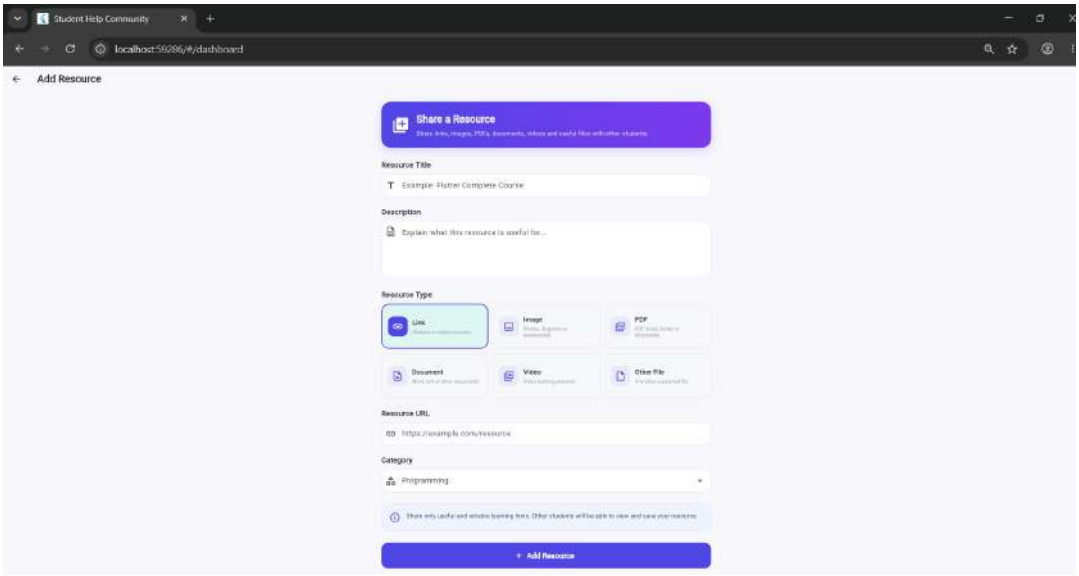
Code

```
final titleController = TextEditingController();
final descriptionController = TextEditingController();
String category = 'Programming';
Column(
  children: [
    TextField(
      controller: titleController,
      decoration: const InputDecoration(labelText: 'Question title'),
    ),
    TextField(
      controller: descriptionController,
      maxLines: 4,
      decoration: const InputDecoration(labelText: 'Describe your question'),
    ),
    DropdownButtonFormField<String>(
      initialValue: category,
      items: const ['Programming', 'Data Science', 'Database']
          .map((x) => DropdownMenuItem(value: x, child: Text(x)))
          .toList(),
      onChanged: (x) { if (x != null) category = x; },
    ),
    ElevatedButton(onPressed: () {}, child: const Text('Post Question')),
  ],
)
```

Output

Capture the Ask Question screen with title, description, category and submit controls visible.





Experiment 7(b): Implement form validation and error handling

Aim

To validate required fields and display clear errors before submitting a question or resource.

Description

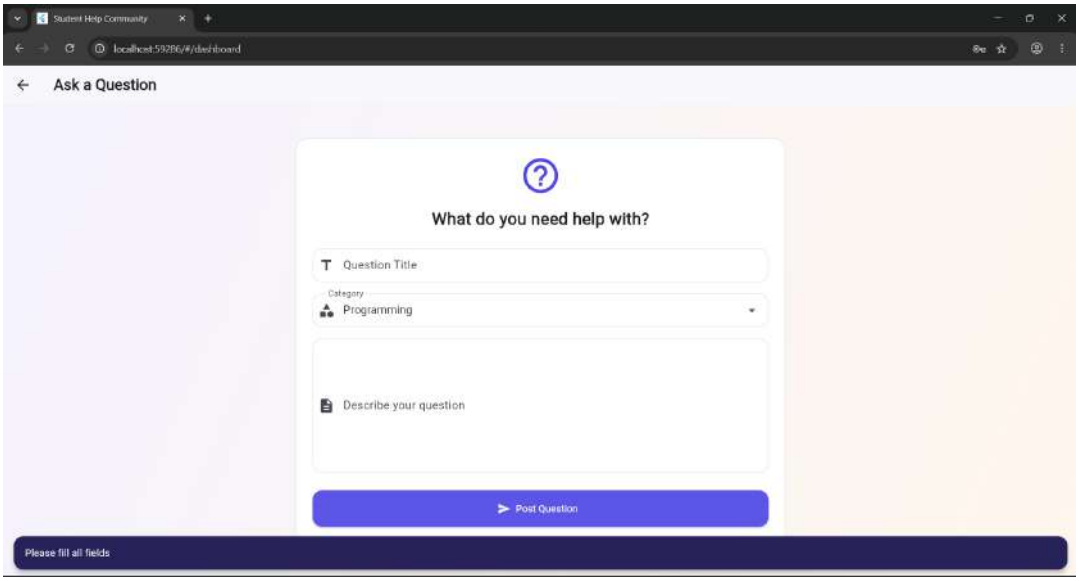
Validation helps prevent empty or malformed submissions.A Form widget and the GlobalKey<FormState> can coordinate validators. A question title can be required and limited to a sensible length; the description can be required. Existing project validation should be documented from the real source. The snippet below is a standard Flutter demonstration.

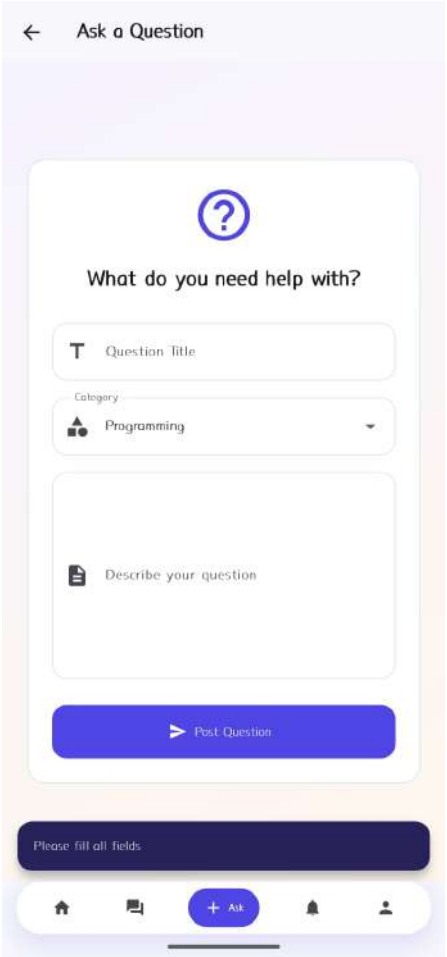
Code

```
final formKey = GlobalKey<FormState>();
final titleController = TextEditingController();
Form(
  key: formKey,
  child: Column(
    children: [
      TextFormField(
        controller: titleController,
        decoration: const InputDecoration(labelText: 'Question title'),
        validator: (value) {
          if (value == null || value.trim().isEmpty) {
            return 'Please enter a question title';
          }
          if (value.trim().length < 5) {
            return 'Enter at least 5 characters';
          }
          return null;
        },
      ),
      ElevatedButton(
        onPressed: () {
          if (formKey.currentState!.validate()) {
            // Continue with the app's real save operation.
          }
        },
        child: const Text('Submit'),
      ),
    ],
  ),
)
```

Output

Capture a validation message after submitting an empty or invalid field.





Experiment 8(a): Add animations to UI elements using Flutter's animation framework

Aim

To demonstrate animation concepts using a Student Help Community splash screen, card or page transition.

Description

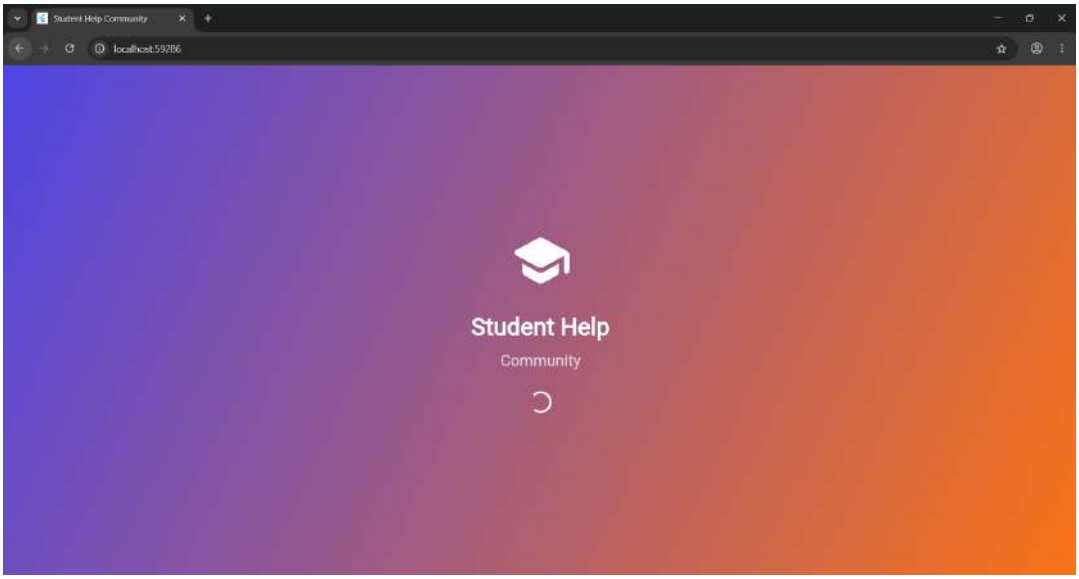
Animations can improve feedback when a screen appears or when a component changes. Flutter offers implicit animation widgets such as AnimatedOpacity and AnimatedContainer, and explicit tools such as AnimationController and FadeTransition. A still screenshot captures only one animation frame; the live app or a short recording is better evidence for motion. Include only effects actually present in the app when describing implemented functionality.

Code

```
AnimatedOpacity(
  opacity: isVisible ? 1.0 : 0.0,
  duration: const Duration(milliseconds: 500),
  curve: Curves.easeInOut,
  child: const Card(
    child: ListTile(
      leading: Icon(Icons.groups_outlined),
      title: Text('Student Help Community'),
      subtitle: Text('Ask, answer and share resources'),
    ),
  ),
)
```

Output

Capture a real animated screen or a representative frame from the splash screen.



Experiment 8(b): Experiment with fade, slide and other animations

Aim

To explore fade and slide animation patterns using student community UI elements.

Description

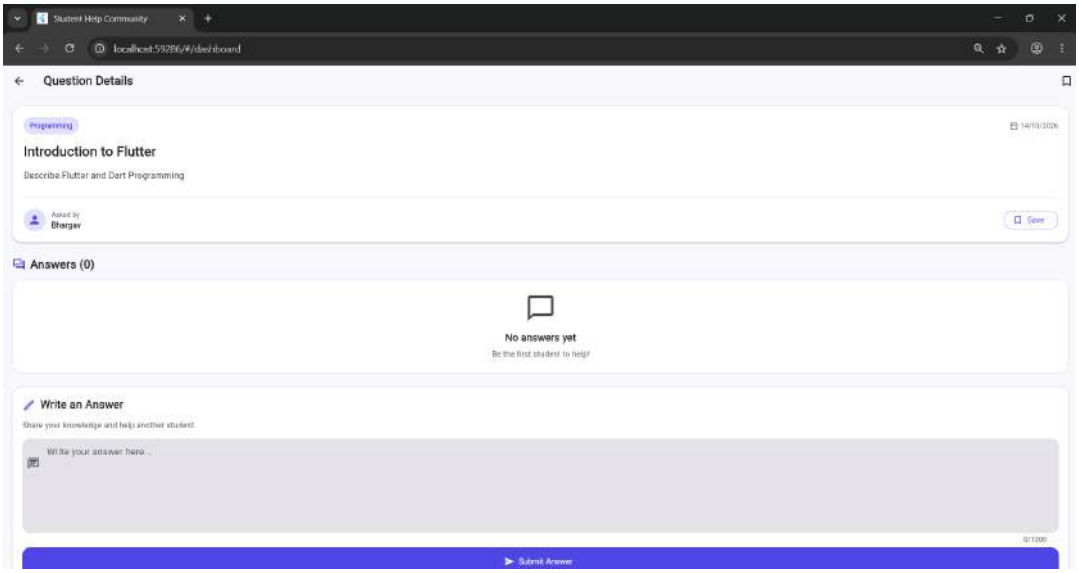
Fade animations change opacity, while slide animations move widgets across a region. They can be used for a notification panel, a question detail transition or an introductory card. AnimationController-based effects require a TickerProvider and must be disposed of correctly.

Code

```
// Inside a State class with SingleTickerProviderStateMixin:
late final AnimationController controller;
late final Animation<double> fade;
@override
void initState() {
  super.initState();
  controller = AnimationController(
    vsync: this,
    duration: const Duration(milliseconds: 450),
  );
  fade = CurvedAnimation(parent: controller, curve: Curves.easeIn);
  controller.forward();
}
// In build():
FadeTransition(
  opacity: fade,
  child: const Text('Welcome to Student Help Community'),
)
@override
void dispose() {
  controller.dispose();
  super.dispose();
}
```

Output

Capture a question details screen and notification screen to demonstrate the movement live.





Experiment 9(a): Fetch data from a REST API

Aim

To understand how a Flutter client requests JSON data from a REST API, using academic resources as the domain example.

Description

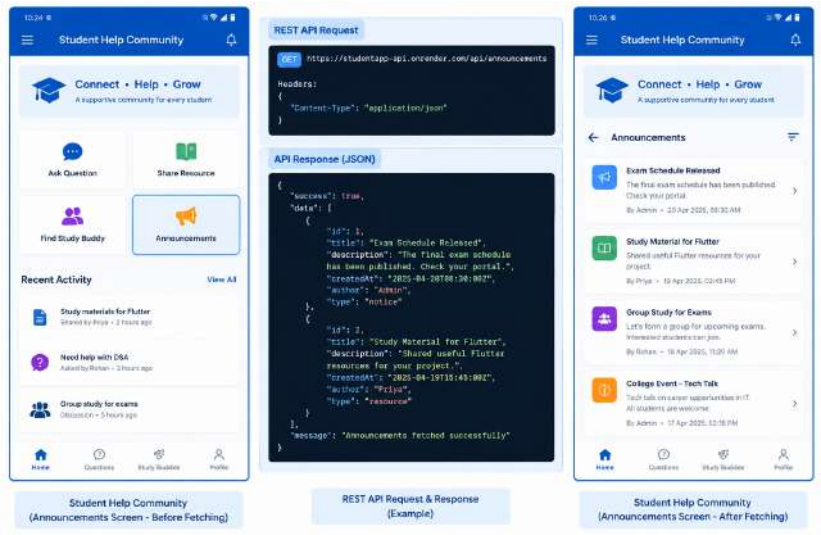
A REST API exposes resources through HTTP endpoints and often returns JSON. A Flutter client can use the http package, check the HTTP status and decode the response. The current Student Help Community is designed around SQLite local data; do not claim its questions or resources are fetched from a remote REST API unless such integration exists.

Code

```dart
// Standalone demonstration; add the http package before use.
import 'dart:convert';
import 'package:http/http.dart' as http;
Future<void> fetchSampleData() async {
  final uri = Uri.parse('https://jsonplaceholder.typicode.com/posts/1');
  final response = await http.get(uri);
  if (response.statusCode == 200) {
    final data = jsonDecode(response.body) as Map<String, dynamic>;
    print('Title: ${data['title']}');
  } else {
    throw Exception('Request failed: ${response.statusCode}');
  }
}
```

Output

Capture the actual terminal or app output showing the fetched sample title and HTTP success.



Experiment 9(b): Display fetched data meaningfully in the UI

Aim

To display data as readable cards or list items using the Student Help Community question/resource interface as a design reference.

Description

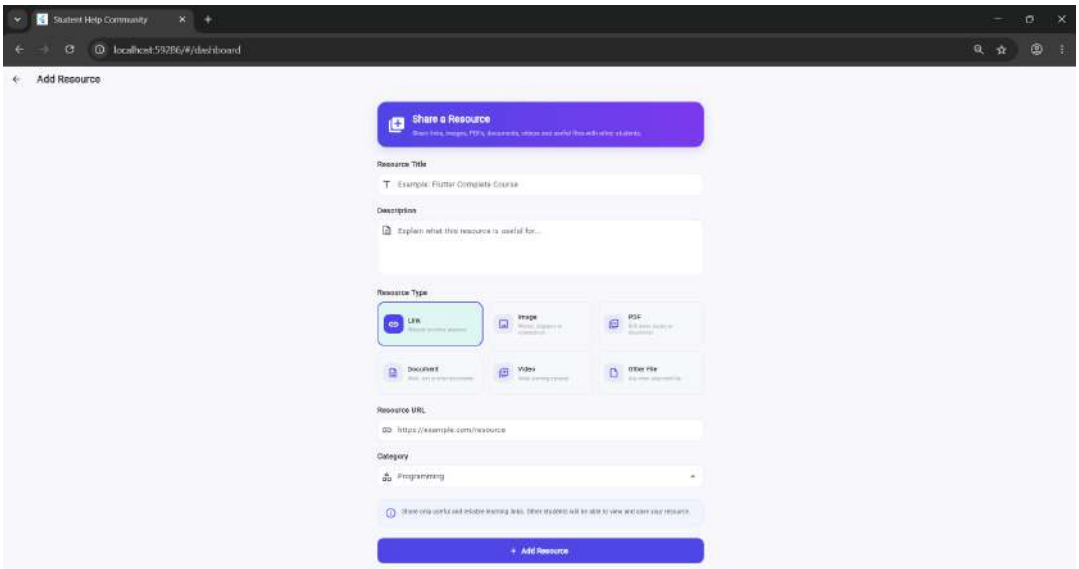
Fetched data becomes useful when presented with meaningful titles, descriptions, category labels and actions. In the existing app, question and resource records are retrieved through local services and displayed in lists/cards. A REST response can use a similar pattern after it has been parsed into a model.

Code

```
ListTile(
  leading: const Icon(Icons.menu_book_outlined),
  title: Text(resourceTitle),
  subtitle: Text(resourceDescription),
  trailing: const Icon(Icons.open_in_new),
  onTap: () {
    // Open the selected resource using the app's actual handler.
  },
)
```

Output

Show the actual Resources screen. If using fetched REST data, make the data source clear; otherwise describe this as the local SQLite list presentation.



Experiment 10(a): Write unit tests for UI components

Aim

To create and run a Flutter widget test for a basic Student Help Community UI component.

Description

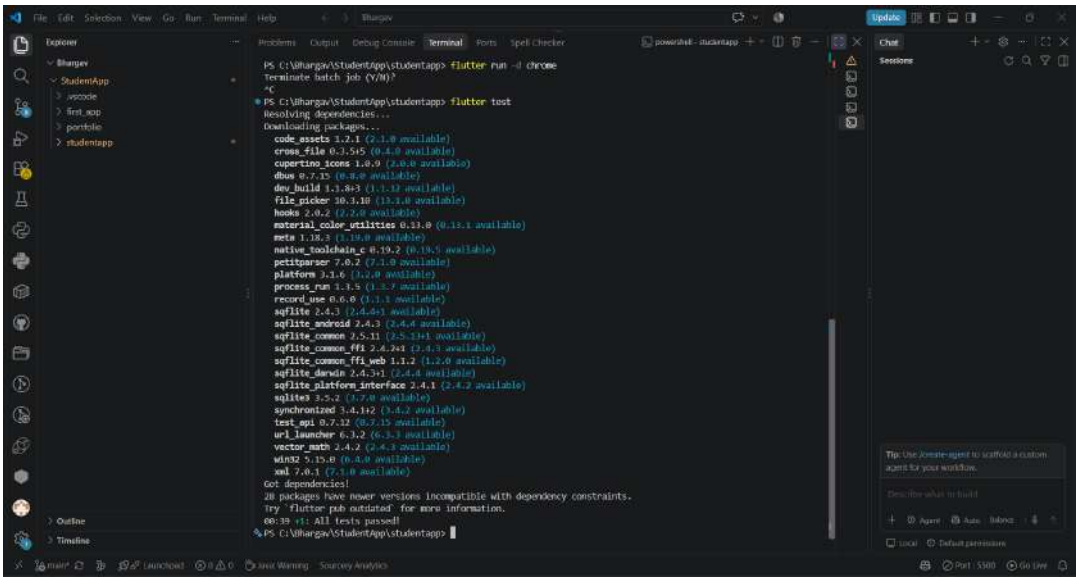
Widget tests verify that a widget renders expected text and reacts as designed. The test below uses a small self-contained widget so it does not depend on the project's database initialization. In a final submission, include the real result from `flutter test`; do not claim tests pass without running them.

Code

```dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
void main() {
  testWidgets('community heading is displayed', (tester) async {
    await tester.pumpWidget(
      const MaterialApp(
        home: Scaffold(
          body: Text('Student Help Community'),
        ),
      ),
    );
    expect(find.text('Student Help Community'), findsOneWidget);
  });
}
```

Output

Run `flutter test` from the app project and capture the actual test summary in the terminal.



Experiment 10(b): Use Flutter debugging tools to identify and fix issues

Aim

To use Flutter's analyzer, debug console and runtime tools to identify issues in Student Help Community.

Description

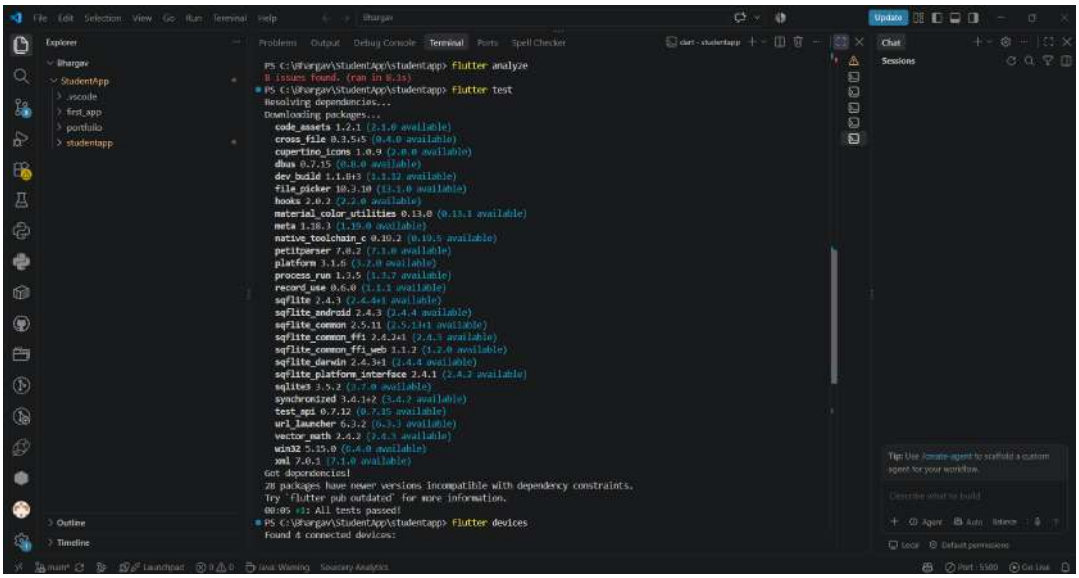
Flutter analyze reports static warnings and errors. Flutter test executes tests, while flutter run and the debug console help identify runtime problems. DevTools can inspect widget layout and performance. A proper record includes the original issue, the correction and the verified final output. Do not state that an issue was fixed unless the new output confirms it.

Code

```
flutter analyze
flutter test
flutter devices
flutter run -d chrome
# Use the device ID shown by flutter devices for Android:
flutter run -d <android-device-id>
```

Output

Capture the real analyzer output and, where applicable, the corrected screen or test result.



Case Study: Student Help Community

Introduction

Student Help Community is a student-oriented Flutter application intended to provide a shared place to ask academic questions, answer peers, share learning resources and save useful material. The UI is organized into user-facing screens and a separate administrator workflow. It uses Dart for application logic and SQLite for local persistence.

Problem Statement

Students often need help understanding course concepts, finding learning materials and keeping useful links organized. A student community interface can place question posting, answers, shared resources and saved content in one application. The project demonstrates these workflows through a local app rather than a centralized production social network.

Objectives

1. Provide user registration and login.
2. Present a dashboard with shortcuts to core functions.
3. Allow students to post questions and read existing questions.
4. Support answers and helpful-answer feedback.
5. Allow learning resources to be shared and browsed.
6. Provide saved resources/content.
7. Display notifications and profile/settings.
8. Show contribution points and a leaderboard.
9. Provide administrator access to inspect and manage community data.
10. Store supported data in SQLite.
11. Demonstrate Flutter UI concepts across Android and Chrome where the web build is configured.

Scope and Main Features

Feature	Description
Authentication	Registration, login and logout for a user account.
Dashboard	Entry point with shortcuts to questions, resources, saved content and profile/community features.
Questions	View question listings, open question details and post a question.
Answers	Read answers and submit an answer to a question.
Resources	Share and browse learning resources; the current source supports link/file-related resource handling as implemented.
Saved content	Save or remove resources and view saved resources.

Notifications	View in-app notifications generated by supported workflows.
Profile	View/update available profile information.
Settings	Manage available preferences, including theme settings if enabled.
Contribution	Earn points for supported community activities.
Leaderboard	Compare contribution totals among users.
Admin	Review user/question/answer/resource data and perform supported management actions.

Technology Stack

Technology	Purpose
Flutter	Cross-platform UI framework.
Dart	Application programming language.
SQLite	Local relational storage.
VS Code / Android Studio	Development, debugging and emulator/device setup.
Android	Mobile build and testing.
Chrome/Web	Browser build and responsive UI testing when web database setup is configured.

System Architecture

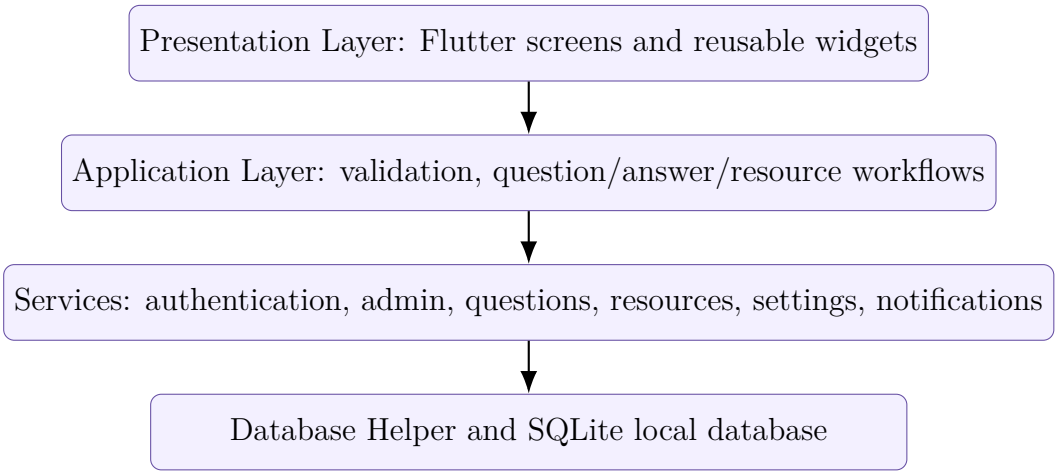


Figure 1: Logical architecture of Student Help Community

Database Design Overview

The project uses a SQLite database accessed through a database helper. The exact schema and columns must be copied from the final `database/database_helper.dart`; the following table describes the main logical data groups known from the project and should be verified against the source before submission.

Table/data group	Purpose
users	Stores account identity and profile data; the project also tracks contribution points.
questions	Stores question content, author reference and question metadata.
answers	Stores answers linked to questions and their authors; helpful state may be recorded.
resources	Stores shared learning resources, author/category and supported link/file metadata.
saved_resources	Associates a user with resources they saved.
settings	Stores supported user preferences.
notifications	Stores in-app notification entries if generated by the current workflows.

Logical relationships

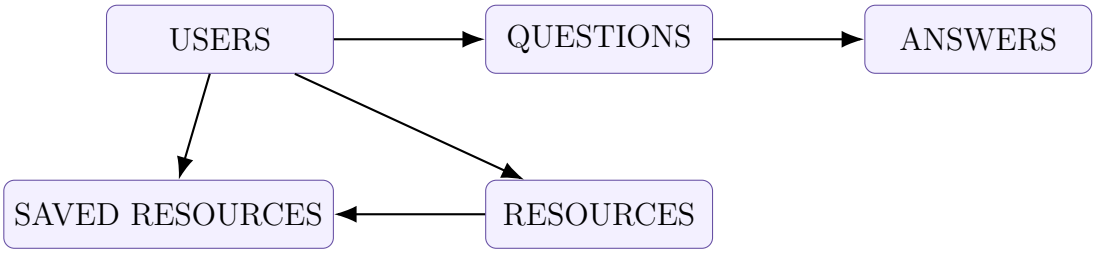


Figure 2: Logical relationships between community data groups

Local storage noteSQLite data on Android and Chrome/Web are separate local stores in this project setup unless an explicit synchronization backend has been implemented. Do not assume that a question added in one platform automatically appears on the other.

User Workflow

1. Launch the app and reach the splash/login flow.
2. Register or log in.
3. Open the dashboard.
4. Browse questions or open Ask Question.
5. Submit a question and inspect it in the question list/detail screen.
6. Open a question and read or add answers.
7. Browse or share learning resources.
8. Save useful resources and open the saved resources screen.
9. Review notifications, profile and settings.
10. View contribution score and leaderboard where available.

Administrator Workflow

1. Open Admin Login from the supported entry point.
2. Authenticate using the configured administrator credentials.
3. Review the administrator dashboard and summary counts.
4. Open users, questions, answers or resources.
5. Search or inspect records as supported.
6. Open detail dialogs/screens and delete records only when appropriate.
7. Return to the overview and logout.

Contribution Points

The implemented point rules recorded for this project are shown below; verify the final service constants before submission.

Activity	Points
Post a question	2
Submit an answer	5
Share a resource	4
Mark an answer helpful	3

Navigation and State

The screens use Flutter navigation and widget state to support interaction. Direct MaterialPageRoute navigation should be documented as such if it is the method present in the source. Named routes and Provider are syllabus concepts and should be described as implemented only after source verification. Stateful widgets can manage local selections, form values and toggles; services and SQLite manage persisted application records.

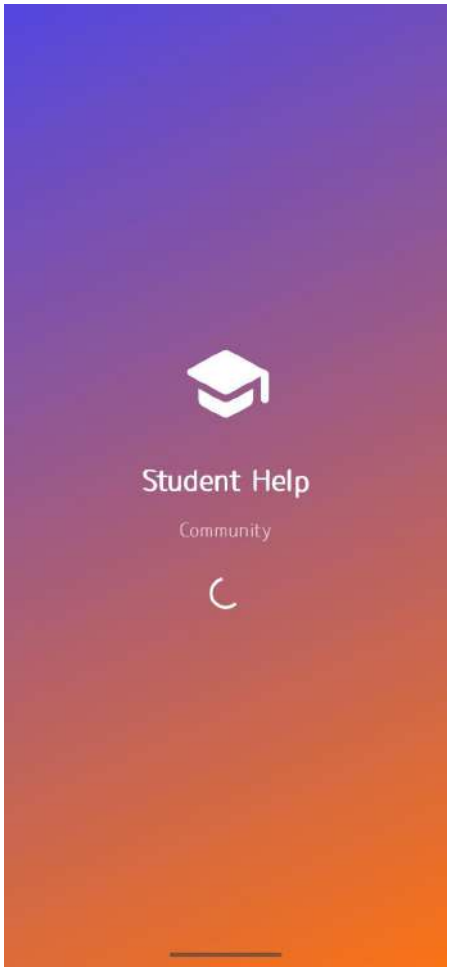
Responsive Android and Chrome Behaviour

Android offers a portrait touch interface, while Chrome can provide a wider viewport and keyboard/mouse input. Screenshots should be captured separately. Local SQLite data is not automatically synchronized between these targets. File picking and other platform capabilities can also differ, so document the actual tested behaviour rather than assuming every feature behaves identically.

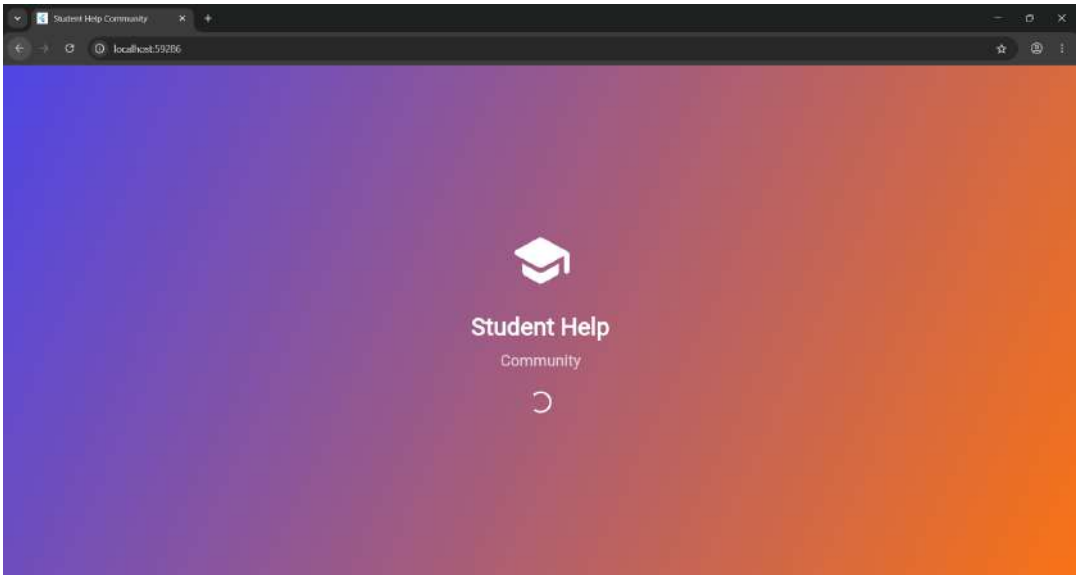
Screen-by-Screen Output Gallery

This section presents the visual outputs of the application through a collection of screenshots. Each screen demonstrates the functionality, user interface, and overall design of the system. Android and Chrome screenshots are included to illustrate the application's appearance across different platforms.

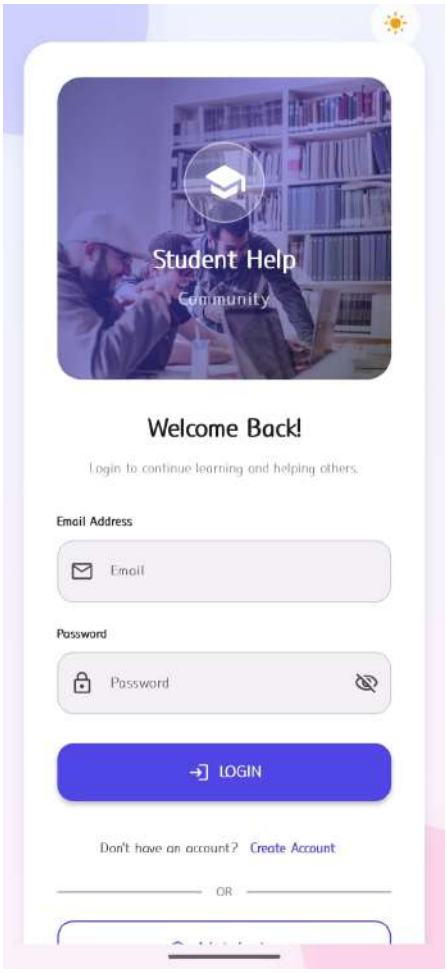
Screen 1: Splash Screen on Android.

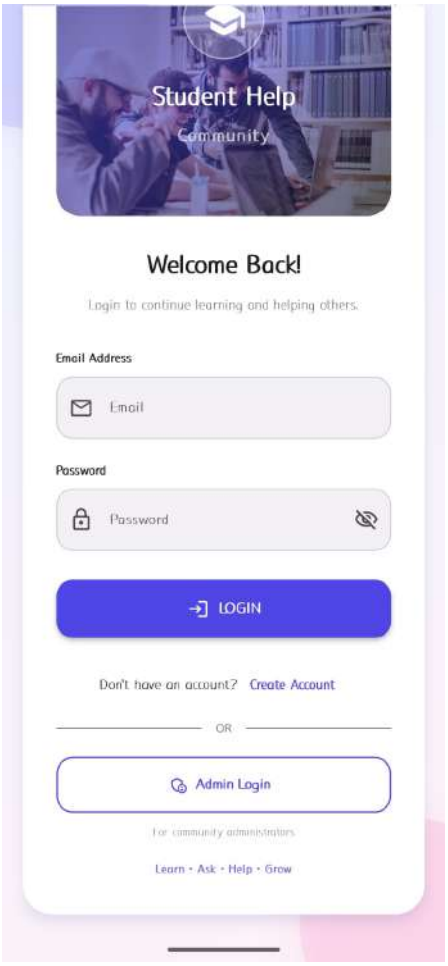


Screen 2: Splash Screen on Chrome.

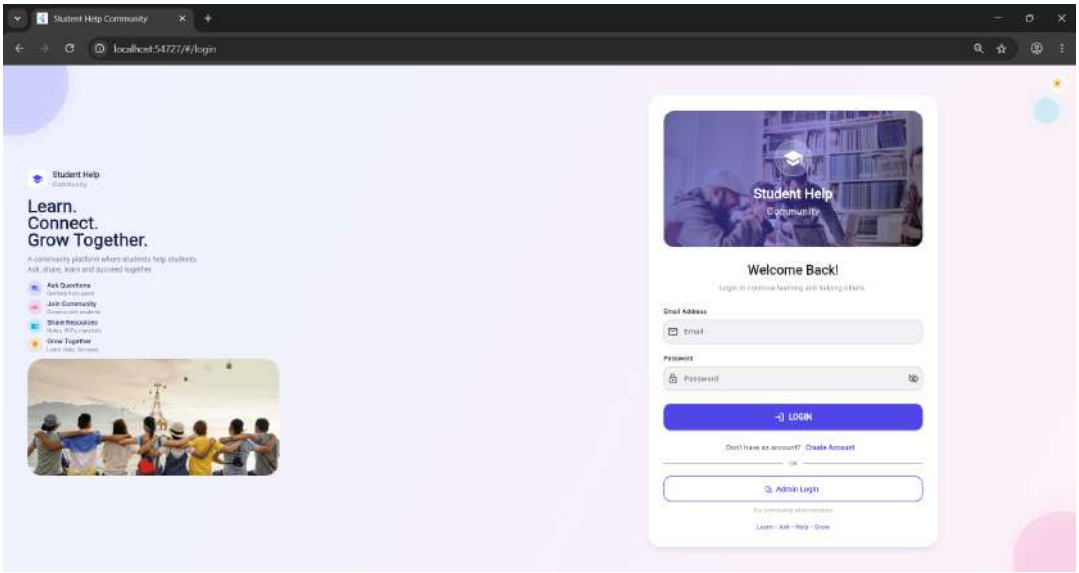


Screen 3: Login Screen on Android.

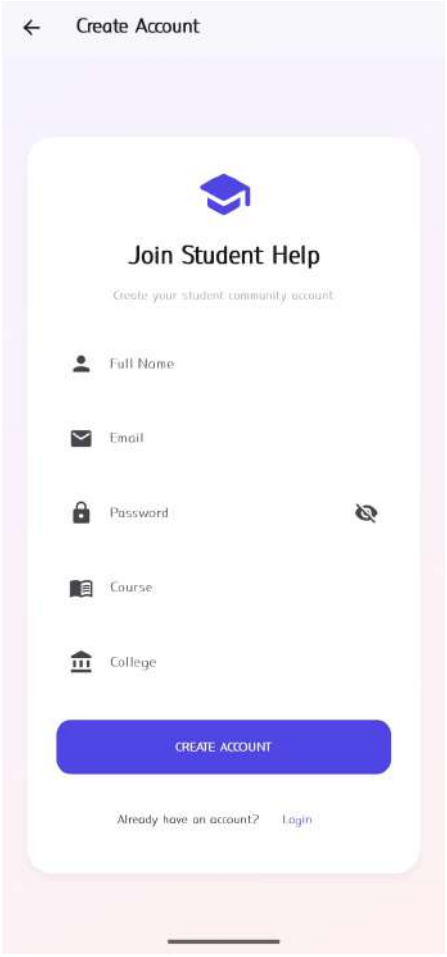




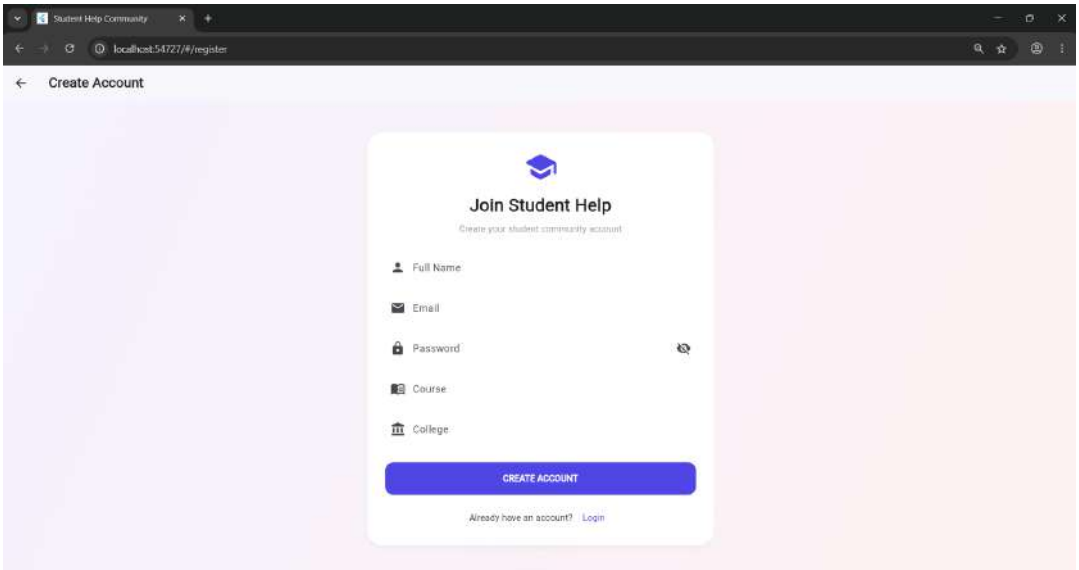
Screen 4: Login Screen on Chrome.



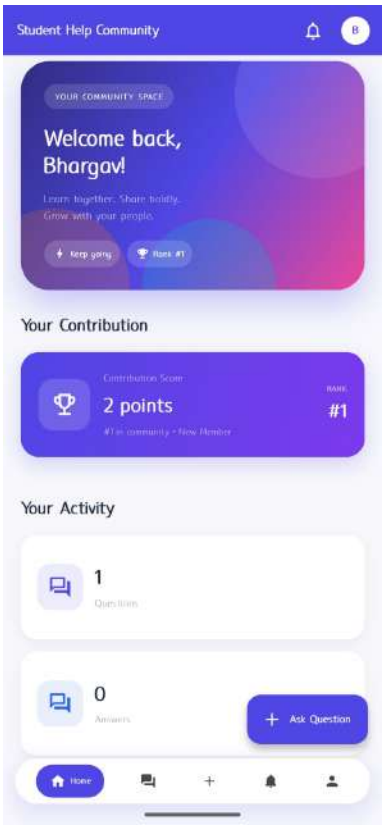
Screen 5: Registration Screen on Android.

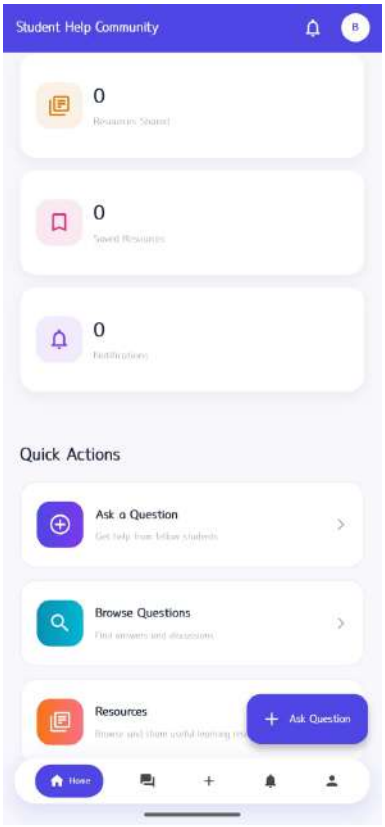


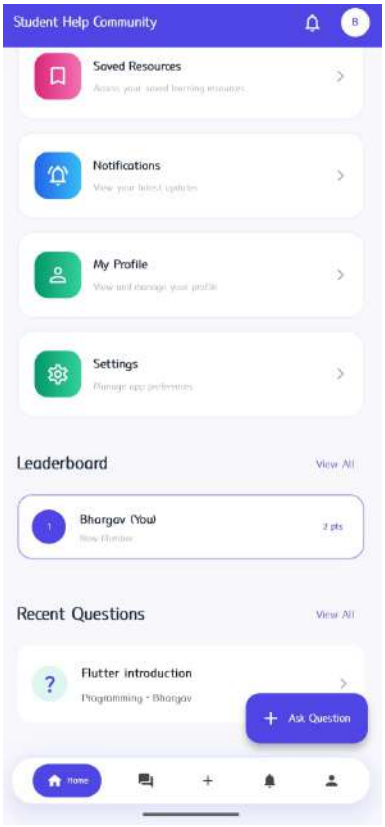
Screen 6: Registration Screen on Chrome.



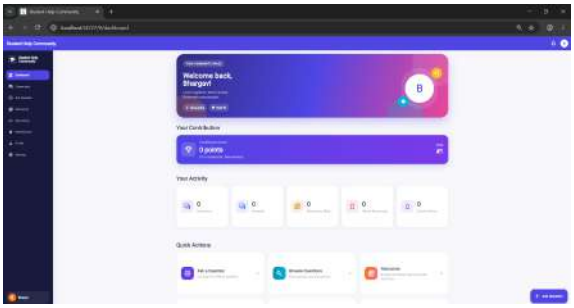
Screen 7: User Dashboard on Android.

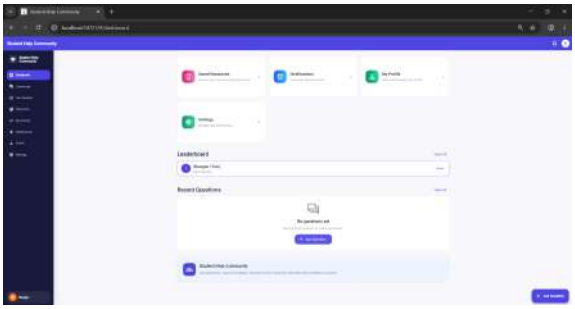




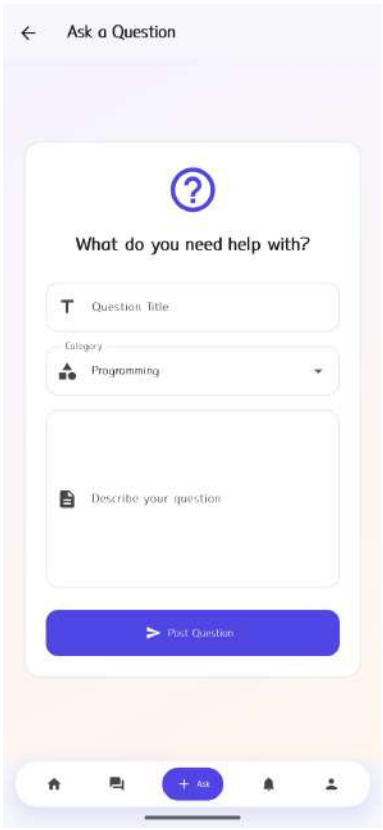


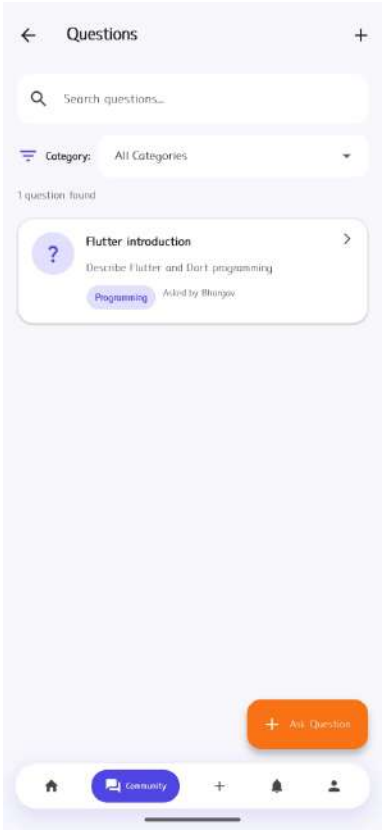
Screen 8: User Dashboard on Chrome.



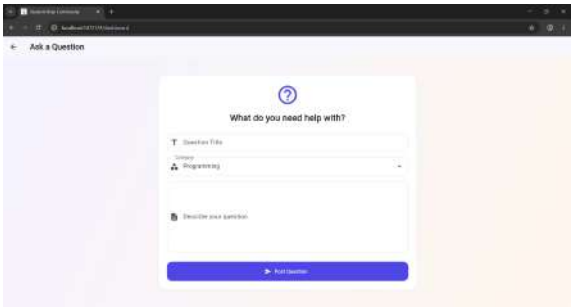


Screen 9: Ask a Question form and Browse Question screen on Android.



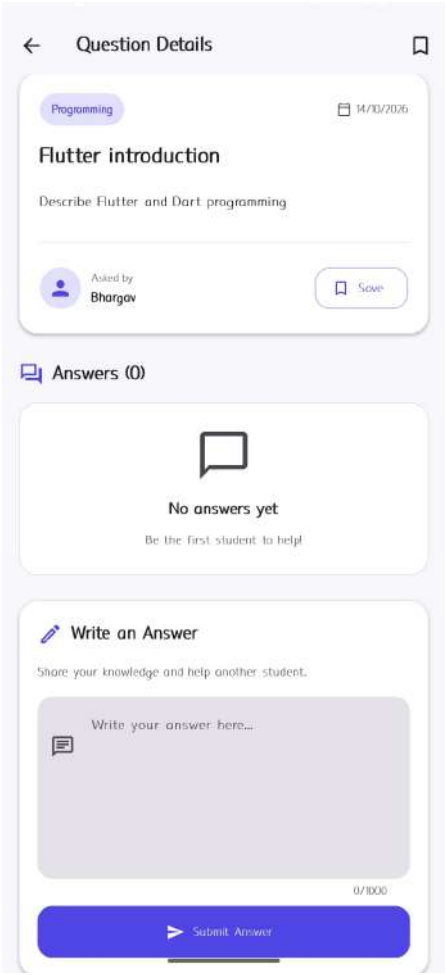


Screen 10: Ask a Question form and Browse Question screen on Chrome.

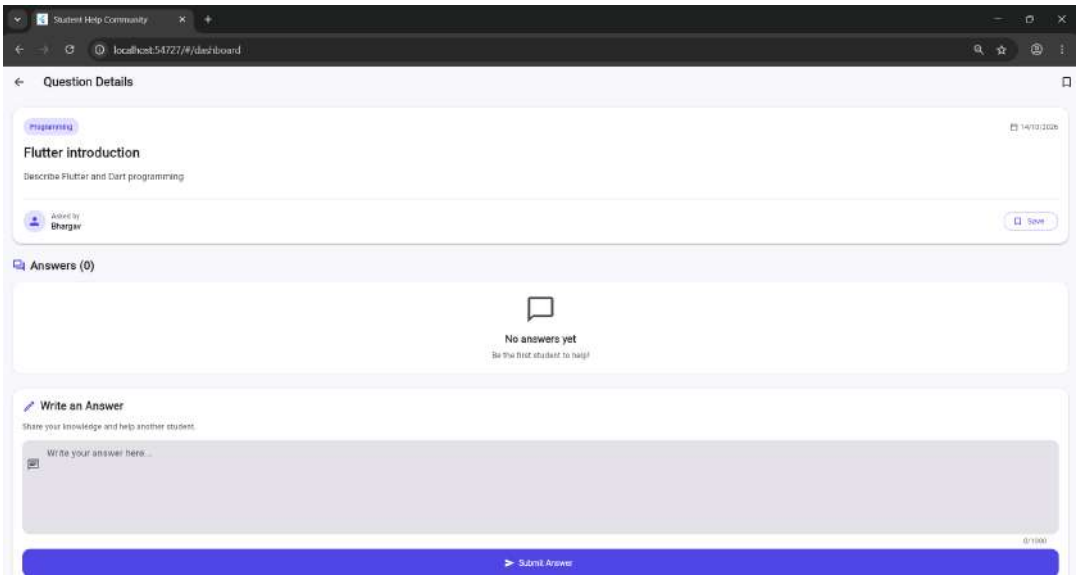




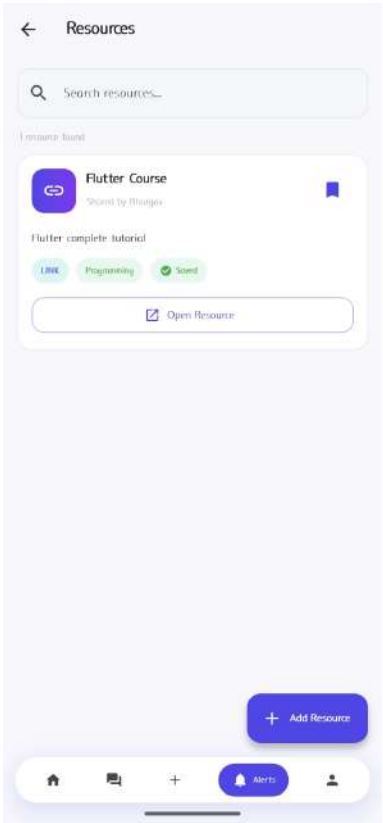
Screen 11: Question Details and Answer submit form on Android.

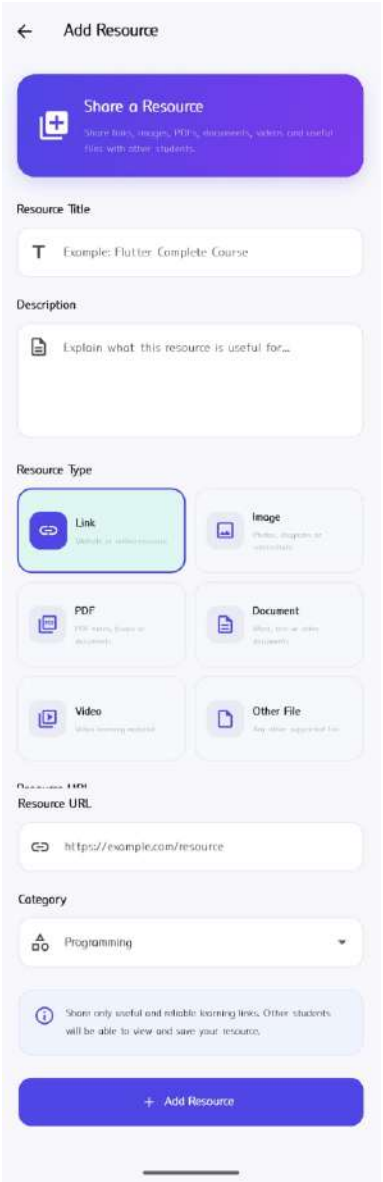


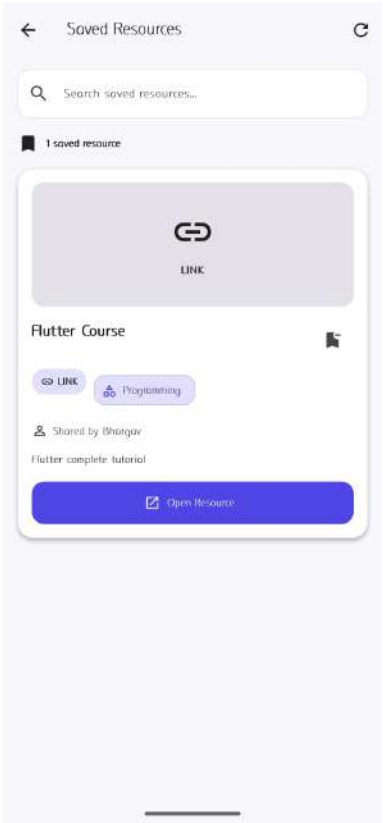
Screen 12: Question Details and Answer submit form on Chrome.



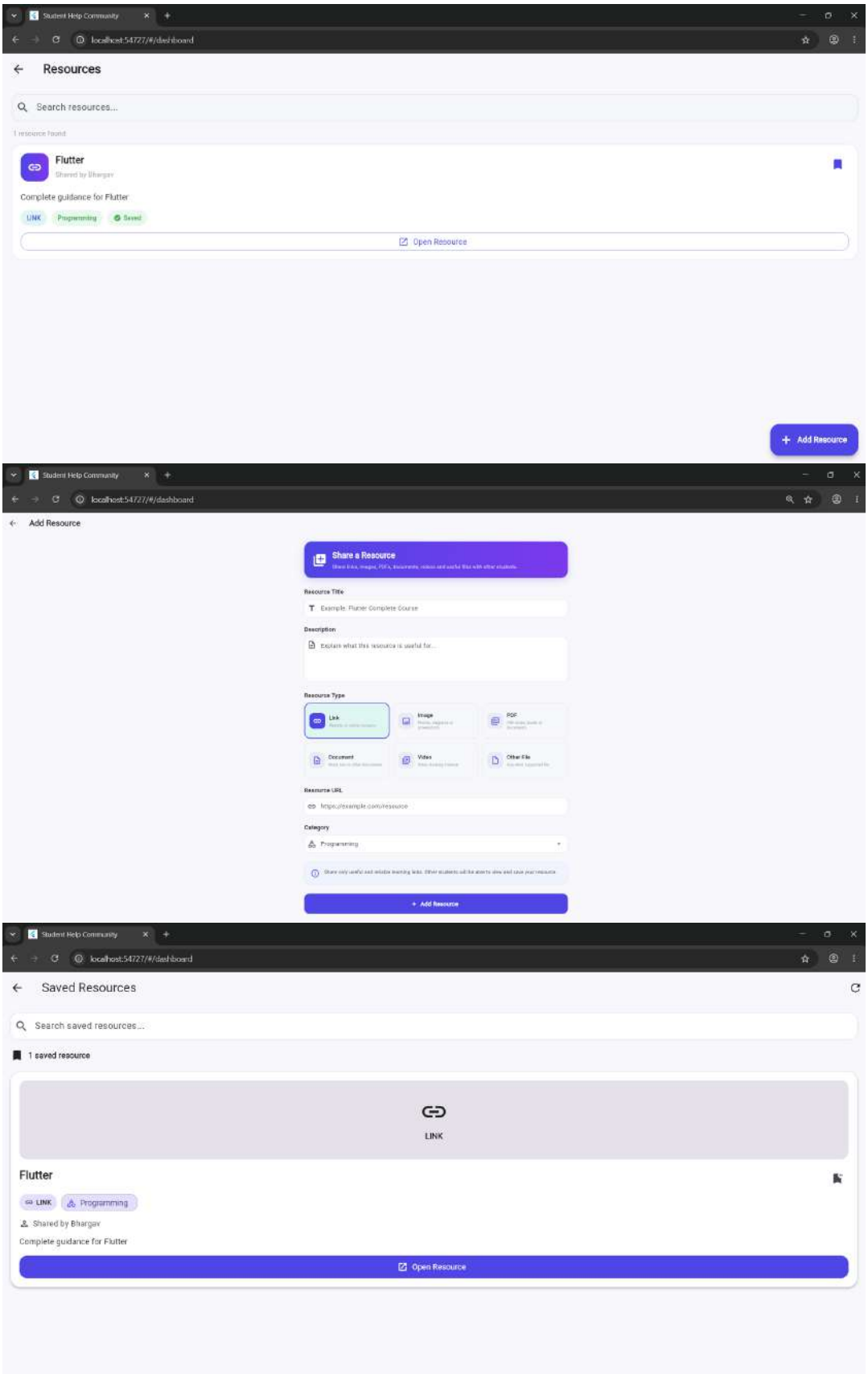
Screen 13: Resources screen, Add Resource form and Saved Resources screen on Android.



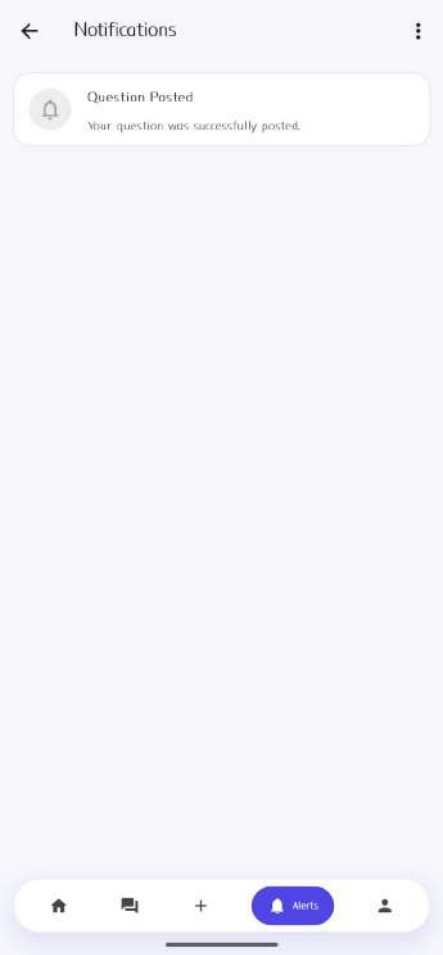




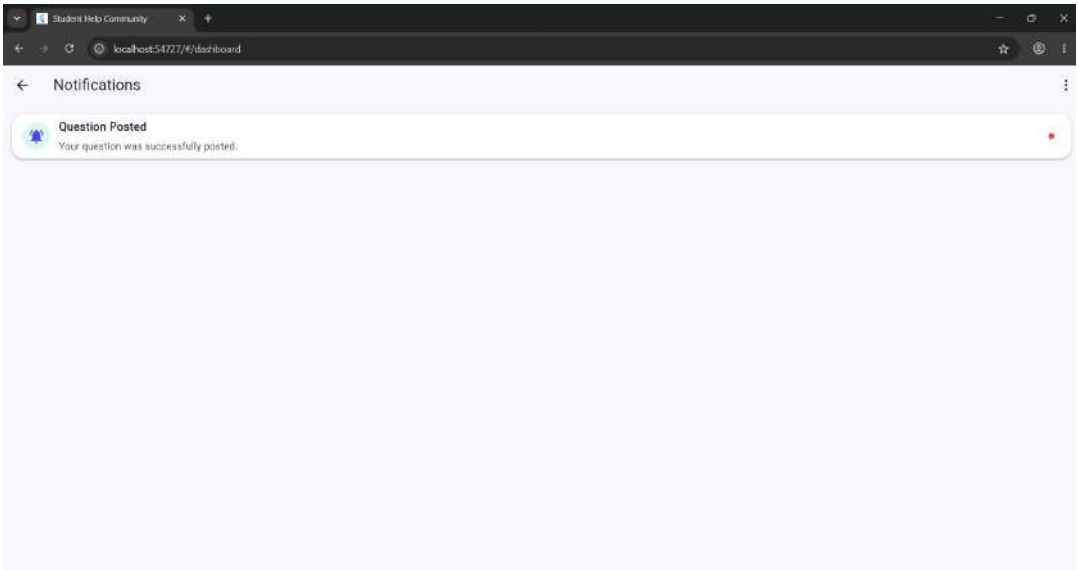
Screen 14: Resources screen, Add Resource form and Saved Resources screen on Chrome.



Screen 15: Notification Screen on Android.



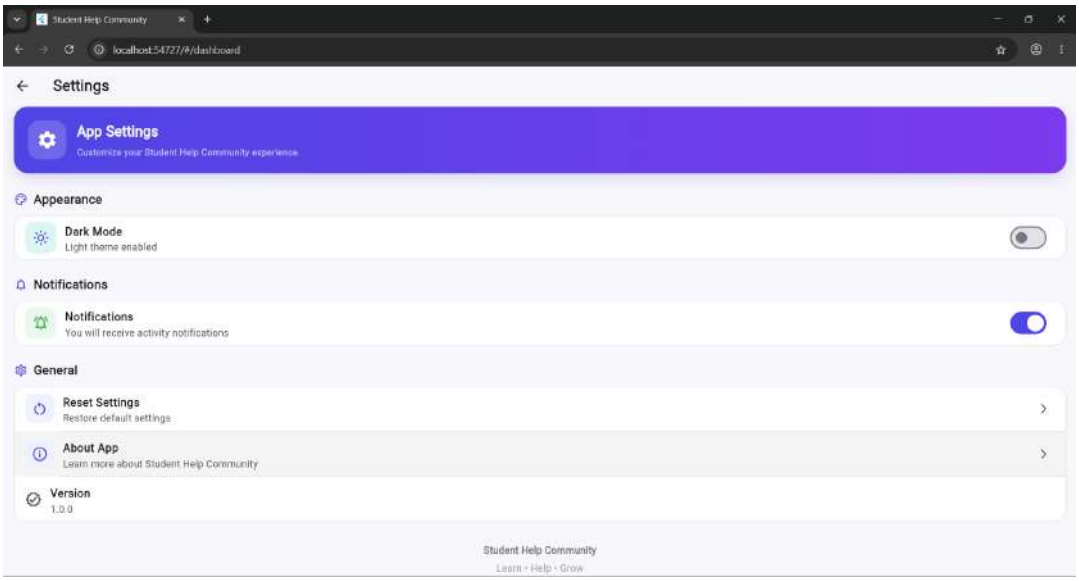
Screen 16: Notification Screen on Chrome.



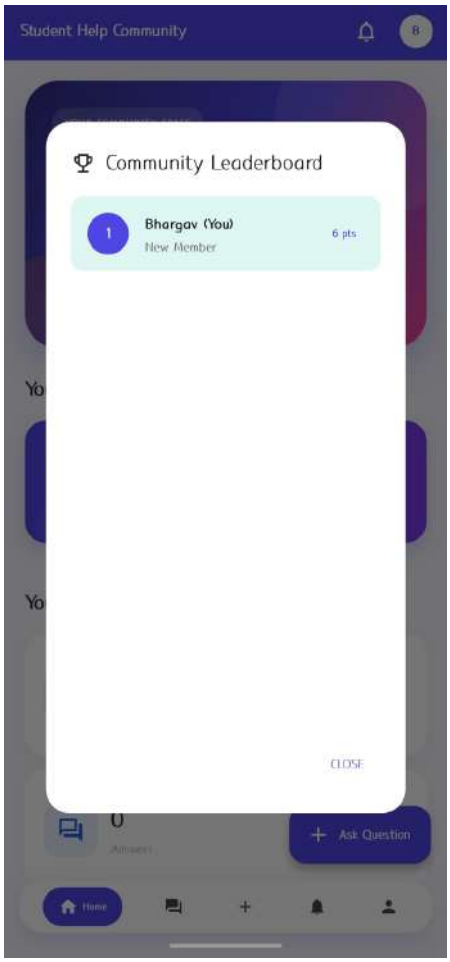
Screen 17: Settings Screen on Android.



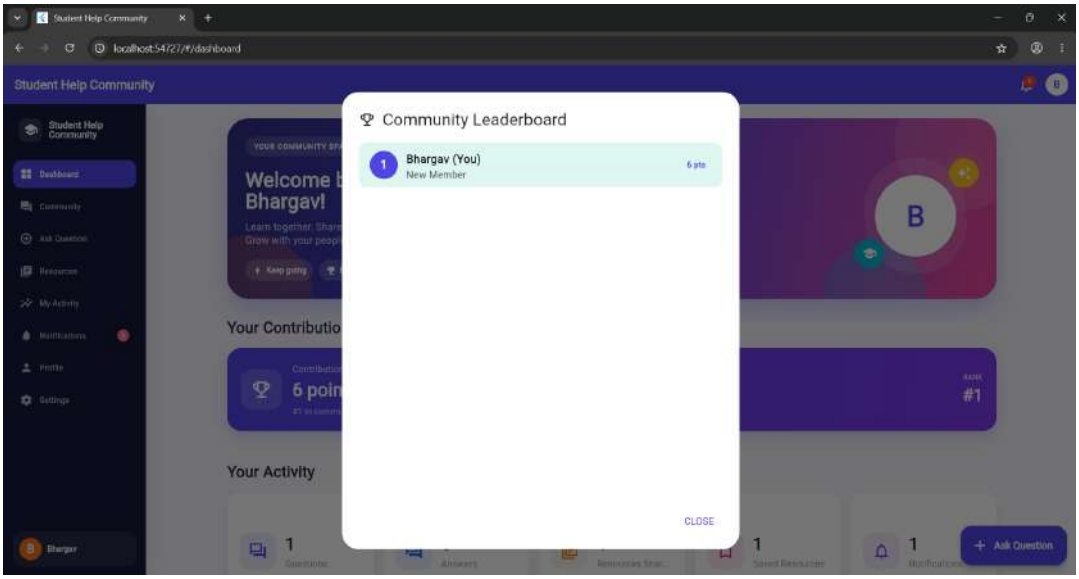
Screen 18: Settings Screen on Chrome.



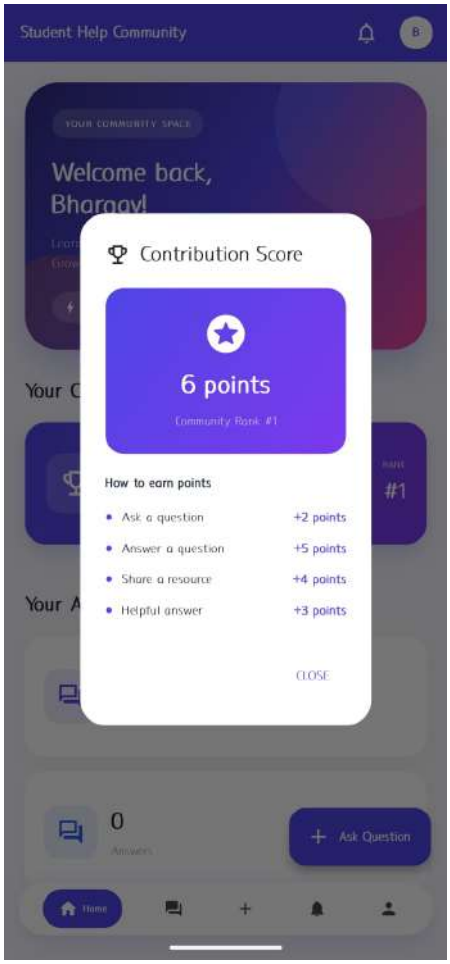
Screen 19: Community Leaderboard Screen on Android.



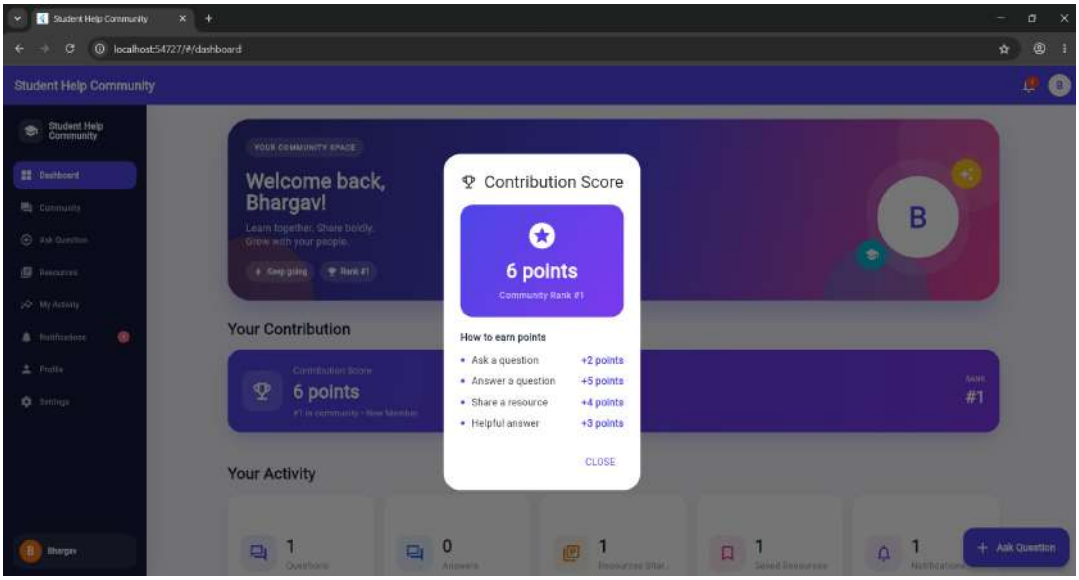
Screen 20: Community Leaderboard Screen on Chrome.



Screen 21: Contribution Score Screen on Android.

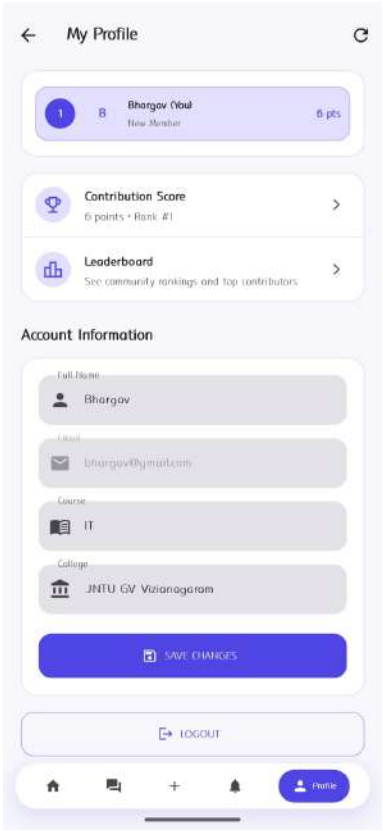


Screen 22: Contribution Score Screen on Chrome.



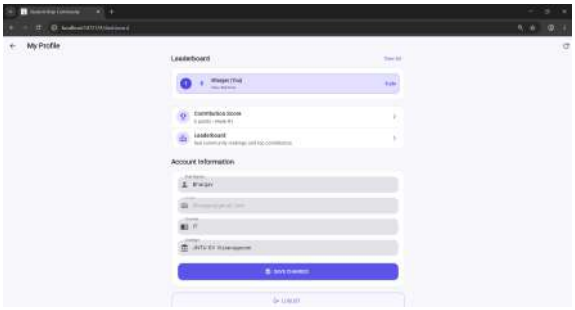
Screen 23: My Profile screen on Android.



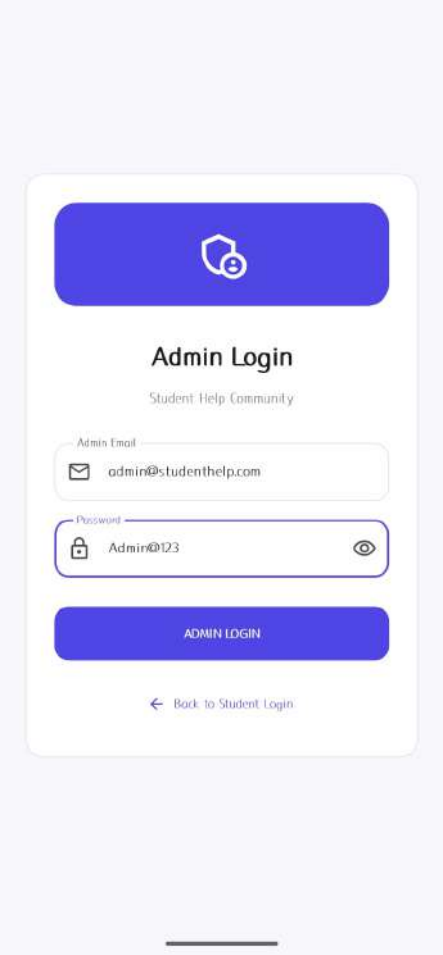


Screen 24: My Profile screen on Chrome.

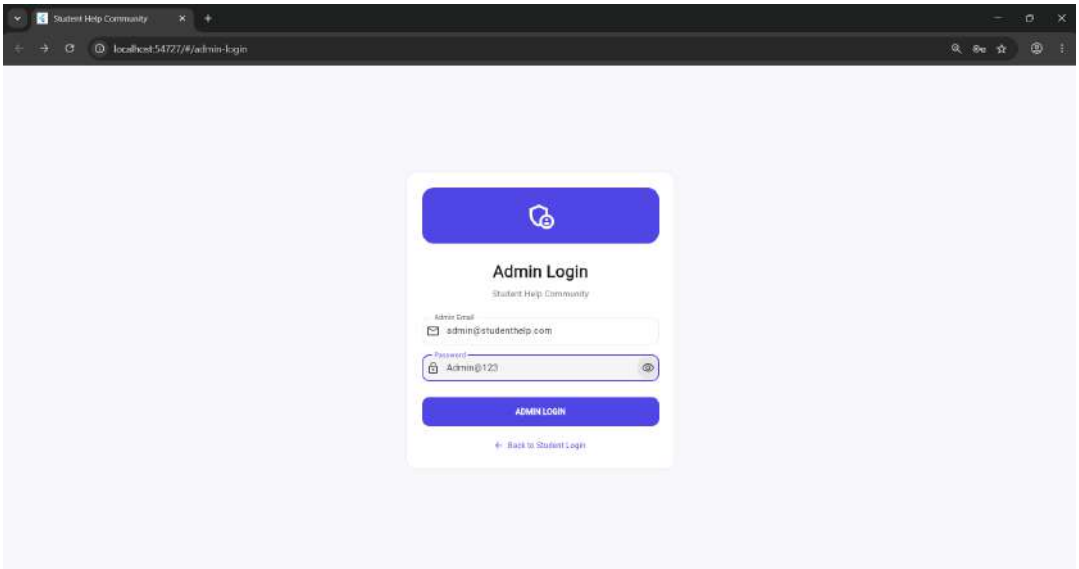




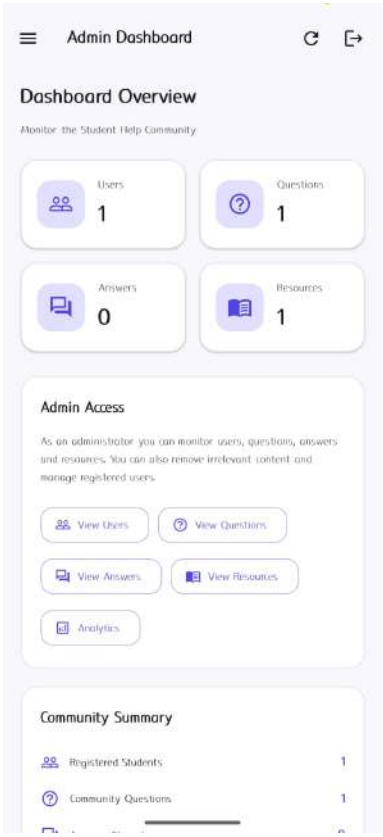
Screen 25: Admin Login Screen on Android.

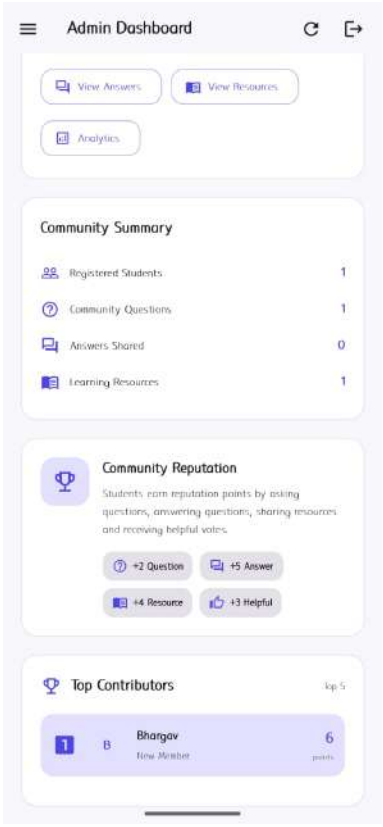


Screen 26: Admin Login Screen on Chrome.

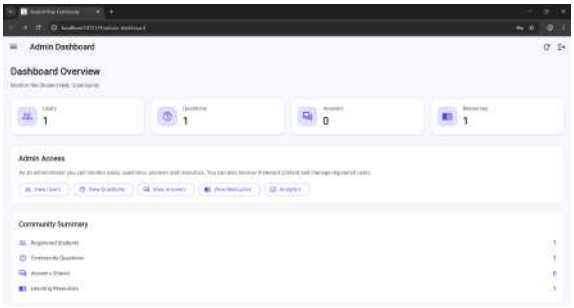


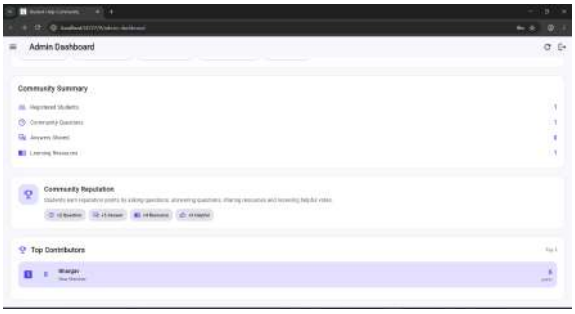
Screen 27: Admin Dashboard on Android.



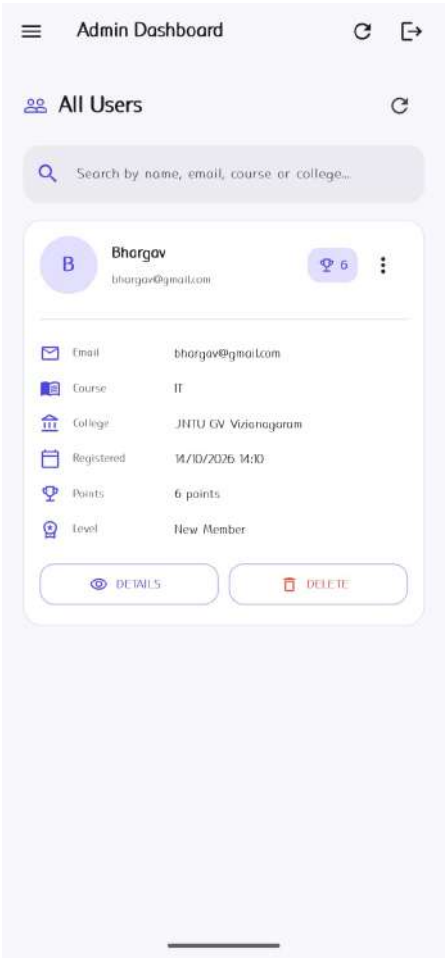


Screen 28: Admin Dashboard on Chrome.

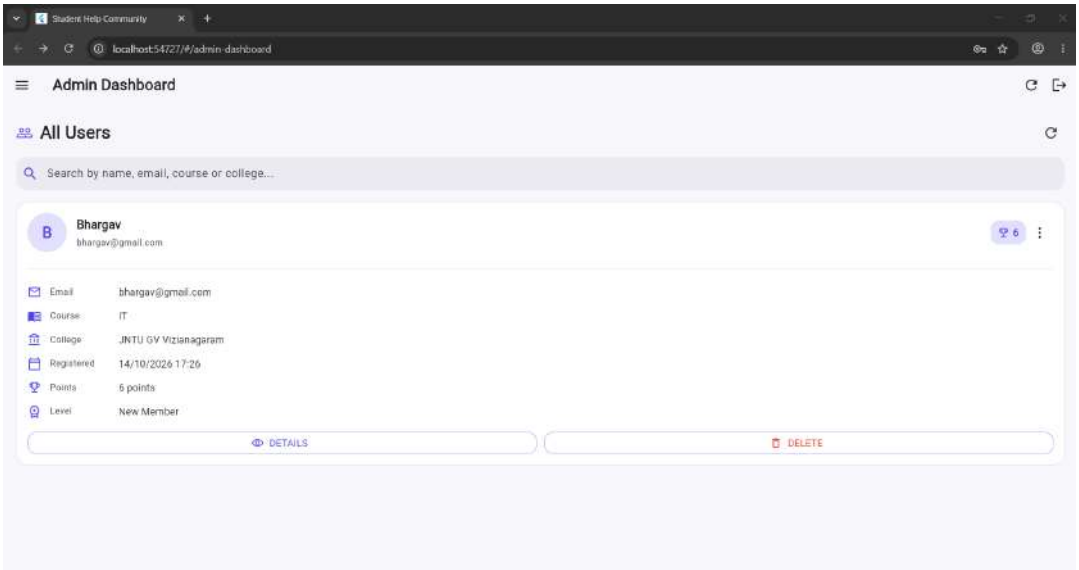




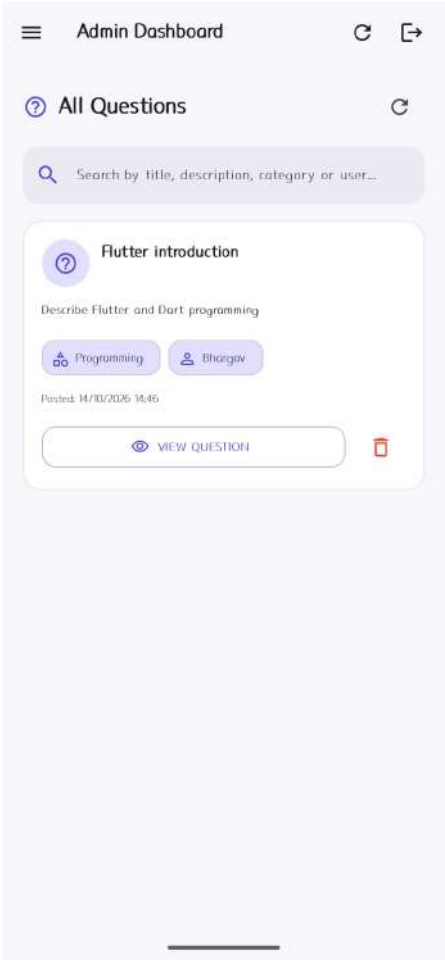
Screen 29: Admin All Users Screen on Android.



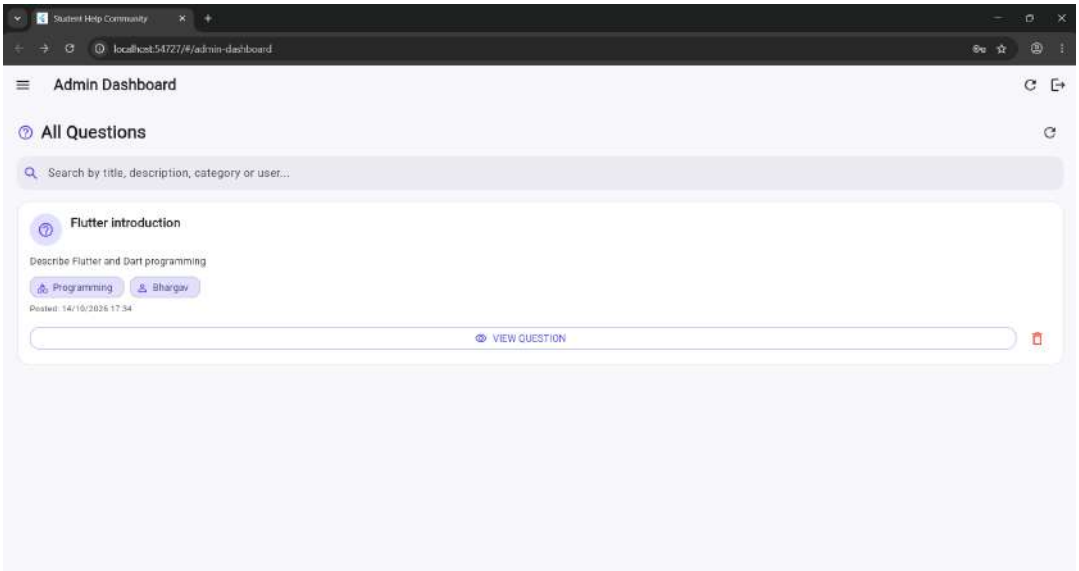
Screen 30: Admin All Users Screen on Chrome.



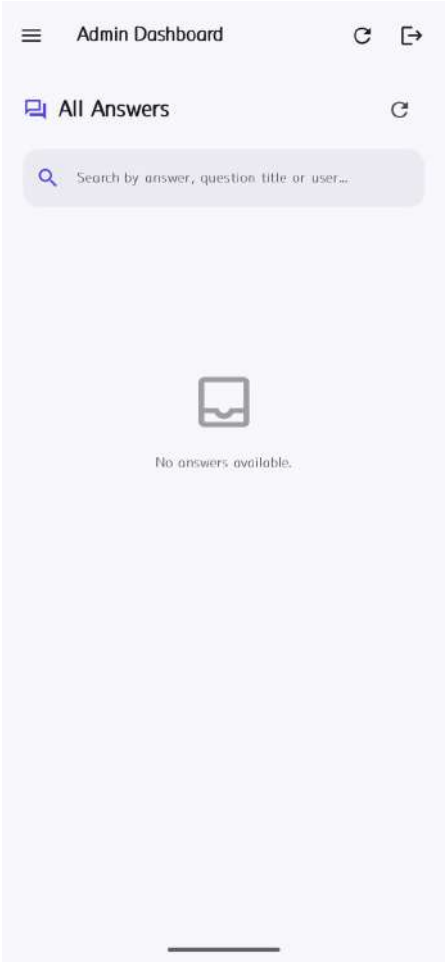
Screen 31: Admin All Questions Screen on Android.



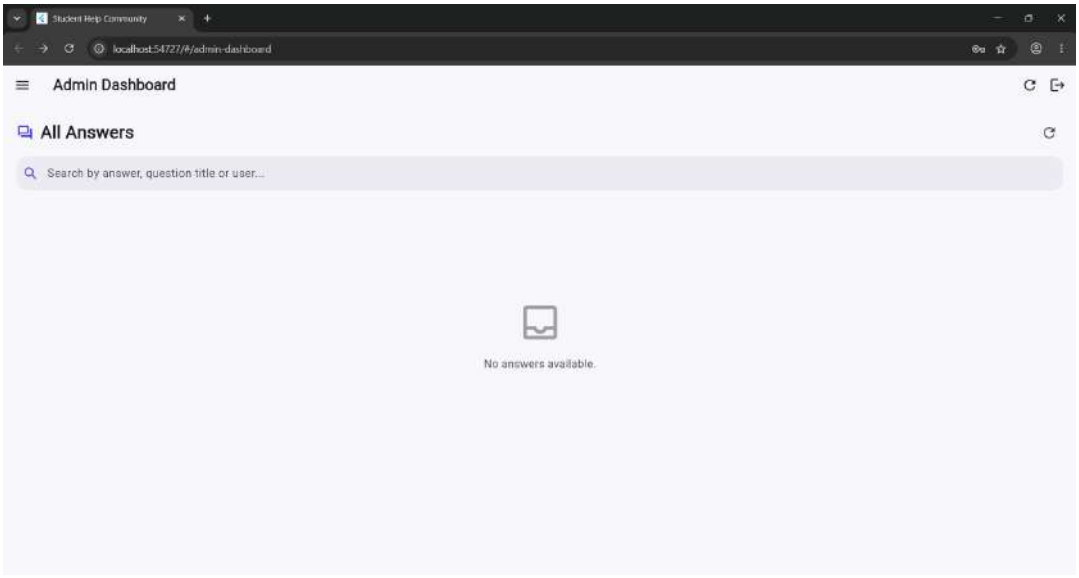
Screen 32: Admin All Questions Screen on Chrome.



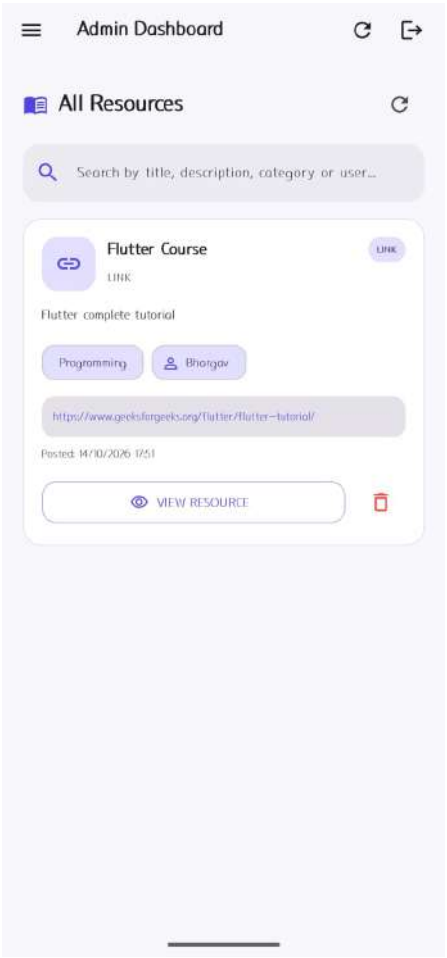
Screen 33: Admin All Answers Screen on Android.



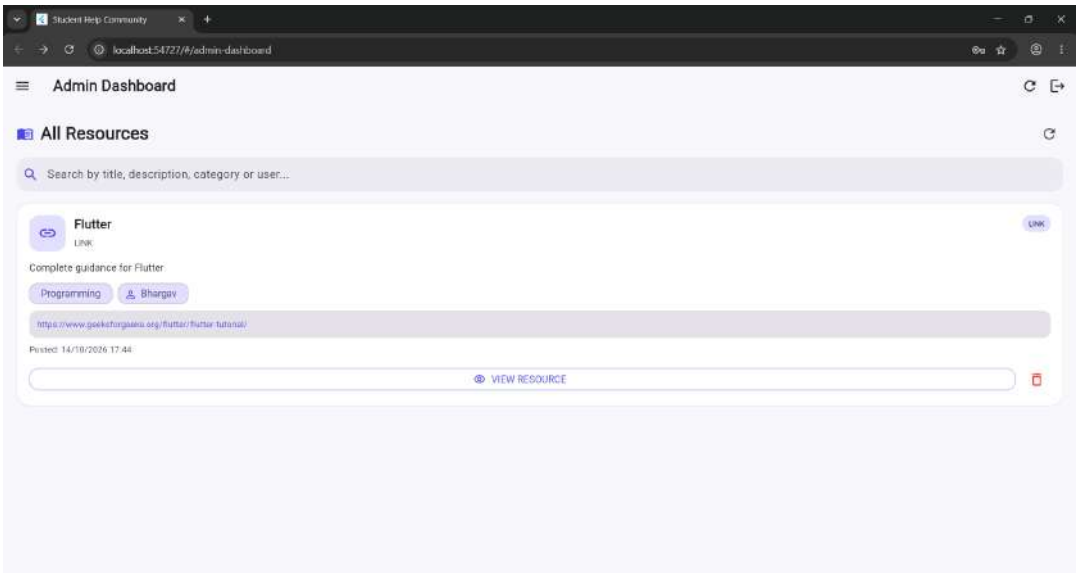
Screen 34: Admin All Answers Screen on Chrome.



Screen 35: Admin All Resources Screen on Android.

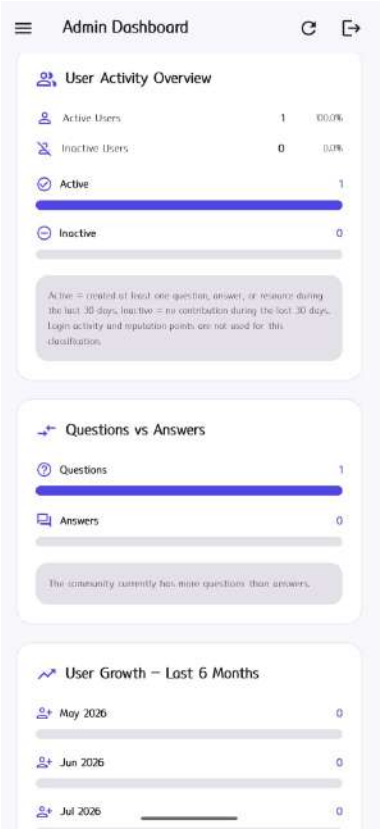


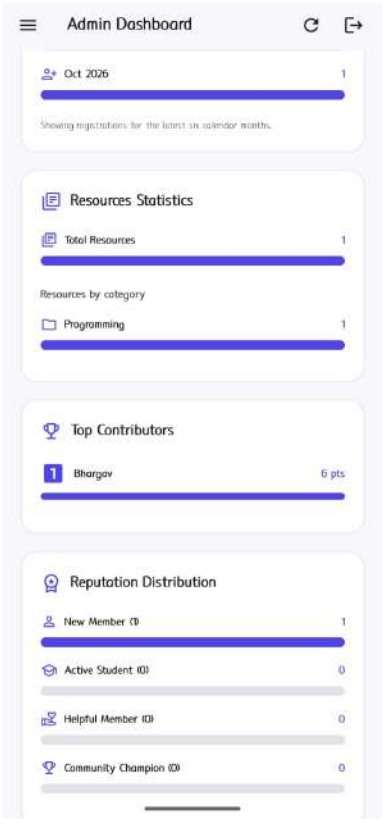
Screen 36: Admin All Resources Screen on Chrome.



Screen 37: Admin Advanced Analytics Screen on Android.

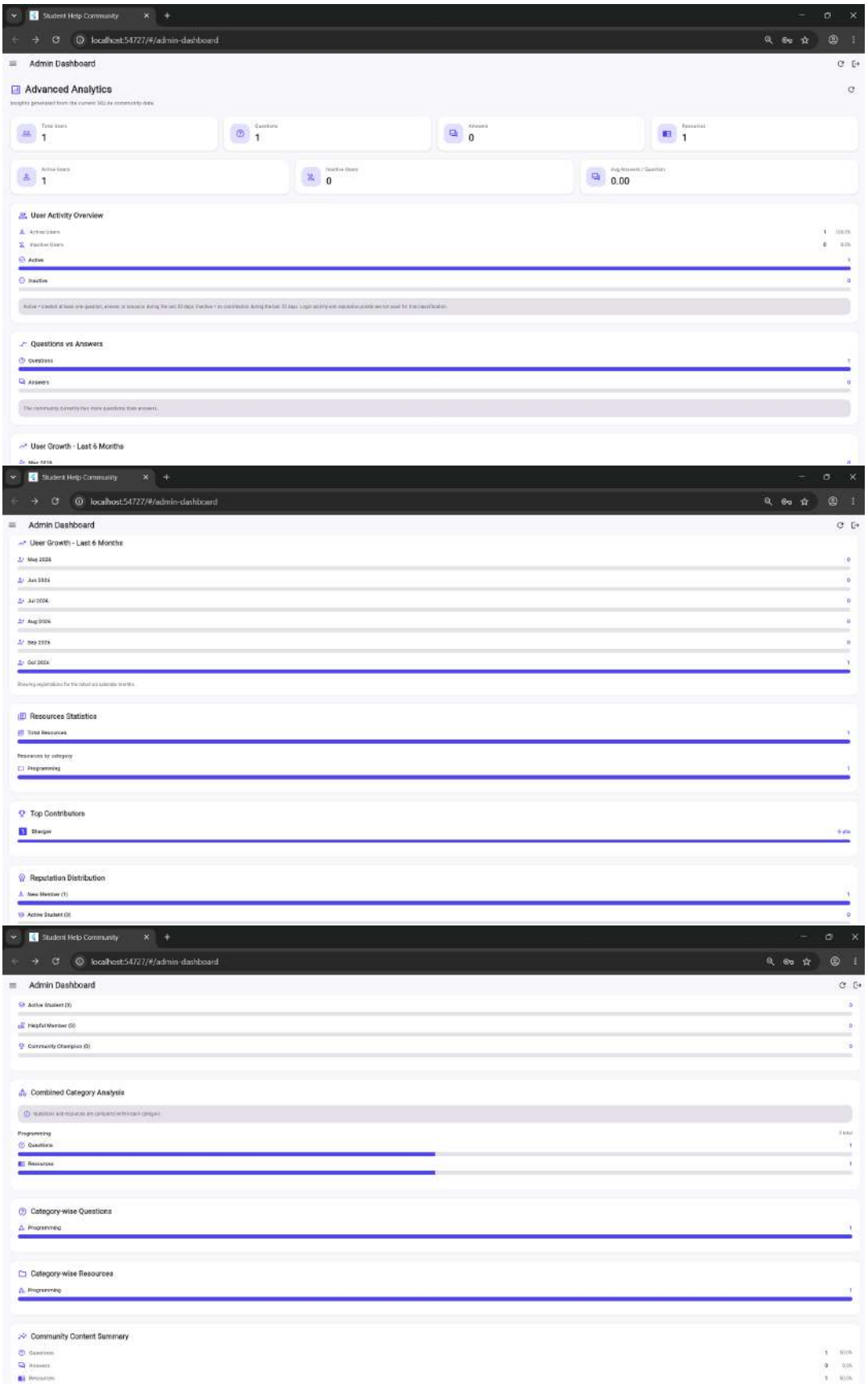








Screen 38: Admin Advanced Analytics Screen on Chrome.



Android and Chrome/Web Comparison

Aspect	Android	Chrome/Web
Screen shape	Usually narrow/portrait; touch interaction.	Browser window can be narrow or wide; responsive layout should adapt.
Image selection	Uses Android-supported picker/permissions.	Uses browser file selection; behavior depends on browser and app implementation.
Location	Requires device permission and available location services.	Requires browser permission and a supported secure context.
Storage	SQLite database on the Android app/device.	Web storage/database behavior depends on the implementation; verify that the chosen SQLite package supports web. Separate platform data stores should not be assumed to sync.
Navigation	Touch/back navigation.	Mouse/touch and browser navigation behavior.

Security and Privacy

Validate user input, keep administrator functionality separate from ordinary user flows, and avoid exposing personal information in screenshots. Do not include real passwords, emails, or private data in the report. For production use, credentials should be stored using a secure password-hashing approach and administrator authorization should not rely only on hiding a button in the UI.

Testing Plan

ID	Test	Expected result	Evidence
T01	Register with valid details	Account creation follows app logic	Register screen
T02	Login with valid credentials	Dashboard opens	Login/dashboard
T03	Invalid login	Error message is shown	Login error
T04	Open question list	Questions are displayed	Questions screen
T05	Submit a question	Saved question appears in list	Ask/Questions
T06	Open question detail	Question and answers are visible	Detail screen

T07	Submit an answer	Answer appears under question	Answer screen
T08	Browse resources	Resource entries are displayed	Resources screen
T09	Save a resource	Saved item appears in saved list	Saved resources
T10	Open notifications	Notification screen loads	Notifications
T11	Open profile/settings	User details/preferences display	Profile/settings
T12	Open leaderboard	Contribution ranking displays	Leaderboard
T13	Admin login	Admin dashboard opens	Admin dashboard
T14	Inspect/delete supported record	Correct record is displayed or removed	Admin detail
T15	Run flutter analyze	Actual warnings/errors are recorded	Terminal
T16	Run flutter test	Actual pass/fail summary is recorded	Terminal
T17	Run on Android	Core screens operate on device/emulator	Android screenshots
T18	Run on Chrome	Web build and UI operate as configured	Chrome screenshots

Limitations

The app is a local student-community implementation rather than a centralized cloud service. Android and browser databases may be separate. Remote REST synchronization, cross-device messaging, production authentication, moderation tools and deployment infrastructure should be listed as future work unless implemented in the final code.

Future Enhancements

- Optional authenticated backend and secure cross-device synchronization.
- Search filters by subject, semester or topic.
- Better moderation and reporting tools.
- Push notifications if a backend is introduced.
- More automated widget and integration tests.
- Accessibility and multilingual improvements.
- Export or backup of learning resources.

Conclusion

Student Help Community demonstrates a practical Flutter-based learning community interface. Its question-and-answer workflow, resource sharing, saved content, notifications, profile/settings, contribution points, leaderboard and administrator area bring several UI design concepts together. The laboratory experiments connect Dart basics, widgets, layouts, responsive design, navigation, state, forms, styling, animation, data display, testing and debugging with the project domain.

References

1. Flutter Documentation, https://docs.flutter.dev/.
2. Dart Documentation, https://dart.dev/guides.
3. SQLite Documentation, https://www.sqlite.org/docs.html.
4. Flutter Testing Documentation, https://docs.flutter.dev/testing.
5. Flutter Navigation Documentation, https://docs.flutter.dev/ui/navigation.
6. Flutter Layout Documentation, https://docs.flutter.dev/ui/layout.
7. Flutter Animation Documentation, https://docs.flutter.dev/ui/animations.
8. Flutter Packages, https://pub.dev/.